INVESTLAB

Documentação de Arquitetura de Software

Arc42 + C4 Model + UML + DER + Software Architecture Canvas

Entrega 2 - Documentação completa (TDE Grupo - PJBL)

Integrantes

Arthur Ambrozewicz Cidral

Erick Marlon Barbieri da Silva

Luan Alberti Estevinho

Vinícius de Oliveira Garcia

Escopo desta entrega: documentação completa da arquitetura do InvestLab, evoluindo o monólito modular da Entrega 1 para uma arquitetura de soluções cloud com *Microfrontend*, *API Gateway*, *Backend for Frontend*, *Microservices* com *Database per Service*, *Serverless* (Azure Functions), *Arquitetura Orientada a Eventos* (Azure Service Bus), *Clean Architecture* + *Vertical Slice* e *Testes Unitários de Arquitetura*. Os níveis 1-4 do C4, UML, DER e Canvas refletem o código do repositório `investlab` que será entregue na Entrega 3.

15 de outubro de 2026

Disciplina: Arquitetura e Soluções Cloud (PSO129) - Prof. Manoel Valerio da Silveira Neto - PUCPR - Eng. de Software - 6º B Noite

Sumário

1. **Introdução e Metas** - Requisitos, metas de qualidade e stakeholders
2. **Restrições de Arquitetura** - Restrições técnicas, organizacionais e convenções
3. **Contexto e Escopo** - Contexto de negócio, contexto técnico e C4 Nível 1
4. **Estratégia de Solução** - Estilos arquiteturais adotados, princípios e Canvas
5. **Visão de Blocos de Construção** - C4 Níveis 2 e 3, UML Nível 4 (componentes e classes) e DER
6. **Visão de Runtime** - Cenários e diagramas UML de sequência (incluindo o fluxo de eventos)
7. **Visão de Implantação** - Azure, AWS API Gateway, MongoDB Atlas, Docker Hub
8. **Conceitos Transversais** - Clean Architecture, Vertical Slice, testes de arquitetura, segurança, eventos, erros, observabilidade
9. **Decisões Arquiteturais** - ADRs com estado (mantidas, substituídas e novas)
10. **Requisitos de Qualidade** - Árvore de qualidade e cenários mensuráveis
11. **Riscos e Dívida Técnica** - Riscos atuais, impacto e mitigação

Referências - Fontes do projeto, material da disciplina e referências oficiais. **Declaração de uso de IA generativa.**

Evolução em relação à Entrega 1

A Entrega 1 (10/09/2026) documentou o InvestLab como um **monólito modular** (React/Next.js + Python/FastAPI + PostgreSQL), deixando os aspectos de nuvem como direção de evolução. Esta entrega mantém **requisitos, metas de qualidade, stakeholders, contexto de negócio, domínio e riscos já aceitos** e substitui as decisões estruturais, conforme a tabela abaixo. Cada substituição está registrada na seção 9 como ADR com estado.

Aspecto	Entrega 1 (parcial)	Entrega 2 (completa)
Decomposição	Monólito modular com 10 componentes internos	2 microserviços + BFF + Azure Functions, cada um com Clean Architecture e fatias verticais
Frontend	Aplicação web única (Next.js)	Microfrontend: shell + 2 remotes (React, Vite, Module Federation) no Azure Static Web Apps
Entrada	Web chama a API diretamente	API Gateway (AWS HTTP API) → BFF (`GET /aggregated-data`) → serviços
Persistência	PostgreSQL único	Database per Service: MongoDB Atlas (ativos) e Azure SQL Database (carteira)
Integração	Chamadas síncronas internas	EDA: evento `cotacao-atualizada` no Azure Service Bus, consumido por Function
Cálculo pesado	Motores dentro do backend	Serverless: simulação em Azure Function (HTTP trigger)
Linguagem	Python (backend) + TypeScript (web)	TypeScript/Node.js em todos os containers (ADR-09)
Garantia estrutural	Convenções	Testes unitários de arquitetura (tsarch, estilo ArchUnit) executados no CI

1. Introdução e Metas

Esta seção apresenta o propósito do InvestLab, os requisitos que orientam a versão 2.0, as metas de qualidade e as partes interessadas. A estrutura segue a seção 1 do arc42 (requisitos relevantes, metas de qualidade e stakeholders).

1.1 Visão geral dos requisitos

O InvestLab é uma plataforma web de inteligência, análise, simulação e comparação de investimentos. A proposta é centralizar informações sobre FIIs, ações e produtos de renda fixa, permitindo análise estatística, simulação de aportes, backtests, comparação de renda fixa e acompanhamento de uma carteira virtual.

O sistema apoia decisão, mas não prevê o futuro nem recomenda automaticamente compra ou venda. As saídas são análises históricas ou simulações baseadas em premissas registradas, deixando a interpretação final sob responsabilidade do usuário.

1.1.1 Problemas que o sistema resolve

- Informações de investimento distribuídas entre diferentes sites e ferramentas.
- Dificuldade para comparar FIIs, ações e renda fixa sob parâmetros comuns.
- Comparações superficiais que consideram apenas rentabilidade bruta.
- Dificuldade para estimar aportes mensais, reinvestimentos e horizontes longos.
- Dificuldade para visualizar risco, volatilidade, drawdown, tributação e liquidez.
- Ausência de um ambiente único para testar e comparar estratégias.

1.1.2 Escopo da versão 2.0

- FIIs, ações, CDB, LCI, LCA e títulos do Tesouro Direto (catálogo de ativos com cotação).
- Pesquisa e visualização de ativos e indicadores; carteira virtual com aportes e patrimônio.
- Simulação de patrimônio com cenários conservador, base e otimista (serverless).
- Reavaliação automática das carteiras quando uma cotação muda (orientado a eventos).
- Dashboard consolidado agregado pelo BFF.
- Preparação para a Entrega 3: Swagger em todos os serviços, imagens Docker, repositórios públicos.

1.1.3 Fora do escopo

- Compra, venda ou custódia de ativos; integração operacional com corretoras ou Open Finance.
- Criptomoedas, mercado internacional, opções e day trade; cotação tick-by-tick.
- Apuração fiscal completa; recomendação automática ou previsão de preços por IA.
- Nesta entrega (documental): autenticação de produção, backtests e laboratório de estratégias ficam como fatias futuras já posicionadas na arquitetura (seção 11.1).

1.1.4 Requisitos funcionais

Os 20 requisitos da Entrega 1 são mantidos; a última coluna indica em qual bloco da nova arquitetura cada requisito é atendido (rastreabilidade requisito → container).

ID	Requisito	Onde é atendido na v2.0
RF01	Criar conta, autenticar e encerrar sessão.	API Gateway (JWT authorizer) - Entrega 3
RF02	Cadastrar e atualizar perfil de simulação.	ms-carteira (fatia futura `perfil`)

RF03	Pesquisar e consultar FIIs, ações e renda fixa.	ms-ativos (`listar-ativos`, `obter-ativo`) via BFF; mfe-ativos
RF04	Exibir preços, proventos e indicadores aplicáveis.	ms-ativos; mfe-ativos
RF05	Calcular retorno, volatilidade, drawdown e estatísticas.	fn-simulacao (função analítica futura)
RF06	Gerar visualizações estatísticas, incluindo Box Plot.	mfe-ativos (fatia futura)
RF07	Simular evolução com capital, aportes, prazo e reinvestimento.	fn-simulacao `POST /api/simular` via BFF; mfe-carteira
RF08	Apresentar cenários conservador, base e otimista.	fn-simulacao (domínio `Simulador`)
RF09	Cadastrar ofertas de CDB, LCI e LCA.	ms-ativos (tipos CDB/LCI/LCA)
RF10	Calcular bruto, tributação estimada, líquido e rentabilidade.	fn-simulacao (fatia futura `renda-fixa`)
RF11	Comparar renda fixa por prazo, liquidez e retorno líquido.	fn-simulacao + BFF (agregação)
RF12	Calcular taxa bruta equivalente.	fn-simulacao
RF13	Informar exposição simulada e cobertura aplicável do FGC.	ms-carteira + fonte FGC
RF14	Executar backtests e comparar com referências.	fn-simulacao (fatia futura `backtest`)
RF15	Criar, editar e acompanhar carteira virtual.	ms-carteira (`criar-carteira`, `registrar-aporte`, `atualizar-precos`); mfe-carteira
RF16	Exibir dashboard consolidado da carteira.	bff-web `GET /aggregated-data`; mfe-shell
RF17	Criar diferentes estratégias no Laboratório.	fatia futura (ms-carteira)
RF18	Comparar estratégias com parâmetros equivalentes.	fatia futura (fn-simulacao + BFF)
RF19	Atualizar dados de mercado periodicamente.	ms-ativos `atualizar-cotacao` + evento `cotacao-atualizada` (Timer trigger na Entrega 3)
RF20	Informar data de atualização e limitações das análises.	Campo `dataReferencia` em todos os contratos; aviso no mfe-shell

1.2 Metas de qualidade

Prior.	Meta	Cenário de avaliação
1	Correção e precisão	Entradas e premissas idênticas produzem resultados determinísticos; fórmulas financeiras corretas e valores monetários em centavos inteiros.
2	Usabilidade	Usuário com conhecimentos básicos conclui uma comparação simples em até 10 minutos, sem treinamento formal.
3	Desempenho	Pelo menos 95% das consultas e análises comuns respondem em até 3 s em condições normais; o dashboard agrega as três fontes em paralelo.
4	Confiabilidade dos dados	Falha de fonte externa não substitui a última base válida; a cotação só avança com data de referência igual ou posterior.
5	Segurança e privacidade	Perfil, carteira e simulações privadas acessíveis apenas a usuários autenticados; serviços nunca expostos sem o gateway; segredos fora do código.

6	Modificabilidade (nova)	Uma funcionalidade nova entra como uma fatia vertical sem alterar outras fatias; a regra das dependências é verificada por testes automatizados.
7	Disponibilidade e resiliência (nova)	A indisponibilidade de um serviço degrada apenas a parte afetada do dashboard; eventos não processados são reentregues e, por fim, enviados à dead-letter.

1.3 Partes interessadas

Stakeholder	Papel	Expectativas
Investidor	Usuário final	Análises compreensíveis, simulações, carteira e comparação sem múltiplas ferramentas.
Equipe de desenvolvimento	Responsável pelo produto	Arquitetura organizada, testável, documentada e viável academicamente; evolução por fatias independentes.
Professor e avaliadores	Avaliação acadêmica	Aplicação correta de Arc42, C4, UML e dos estilos arquiteturais da disciplina, refletidos no código.
Provedores de dados (B3, CVM, BCB, Tesouro, FGC)	Sistemas externos	Disponibilidade de dados de mercado, fundos e indicadores dentro das condições de uso.
Responsável pela manutenção	Operação técnica	Monitorar integrações e filas, corrigir falhas, controlar custos dos planos gratuitos.

2. Restrições de Arquitetura

As restrições abaixo delimitam as escolhas do projeto. As marcadas como *novas* decorrem dos estilos arquiteturais exigidos na disciplina e dos planos gratuitos disponíveis à equipe.

2.1 Restrições técnicas

ID	Restrição	Consequência
RT01	Aplicação web responsiva	Acesso por navegador em computadores e dispositivos móveis.
RT02	React + Vite + Module Federation	Frontend dividido em shell e remotes implantáveis de forma independente (substitui Next.js).
RT03	Node.js 22 + TypeScript em todos os serviços	Uma única linguagem para BFF (exigido pelo enunciado), microserviços e Functions; menor carga cognitiva para a equipe.
RT04	MongoDB Atlas (free tier) e Azure SQL Database (free, 1 DTU)	Database per Service: cada microserviço é dono exclusivo da sua base; nenhuma consulta cruza bases.
RT05	Microserviços + BFF + API Gateway	O frontend só conhece o gateway; o gateway só conhece o BFF; o BFF orquestra serviços e Functions.
RT06	REST + JSON + HTTPS; eventos JSON versionados	Contratos síncronos em OpenAPI 3 (Swagger) e assíncronos em JSON Schema (`contracts/events`).
RT07	Dependência de fontes externas	Mudanças ou indisponibilidade afetam atualização; integrações isoladas no ms-ativos.
RT08	Resultados com data e premissas	Evita apresentar cenários hipotéticos ou dados desatualizados como fatos atuais.
RT09	Sem execução de ordens	O sistema permanece analítico, sem movimentação financeira.
RT10	Markdown e Mermaid no material-base	Documentação versionada junto com o código; este PDF é gerado a partir de `docs/`.
RT11	Azure for Students e planos gratuitos (nova)	Functions no plano Consumption, Service Bus Basic/Standard mínimo, SWA Free, Atlas M0, Azure SQL free; limites de carga e sem SLA.
RT12	Repositórios públicos no GitHub e imagens no Docker Hub (nova)	Nenhum segredo no código ou nas imagens; configuração exclusivamente por variáveis de ambiente.

2.2 Restrições organizacionais

ID	Restrição	Consequência
RO01	Mesma equipe do TDE (4 integrantes)	Mantém continuidade do projeto desde a Entrega 1.
RO02	Recursos acadêmicos	Funcionalidades priorizadas; complexidades podem ser postergadas como fatias futuras.
RO03	Entrega documental completa (Entrega 2) e de software (Entrega 3, 12/11/2026)	Esta versão documenta os 11 capítulos; o código entregue na Entrega 3 deve refletir esta documentação.
RO04	Versionamento	Código e documentação com rastreabilidade (ADRs, contratos versionados, figuras numeradas).

RO05	Uso autorizado das fontes e da IA generativa	Dados externos respeitam condições de uso; uso de IA declarado (Res. 274/2024 CONSUN).

2.3 Convenções

ID	Convenção	Aplicação
CV01	RFNN / ADR-NN / QNN / RNN	Identificadores de requisitos, decisões, cenários de qualidade e riscos.
CV02	REST / JSON	Recursos no plural (`/ativos` , `/carteiras`), verbos HTTP semânticos, erros padronizados (`{erro, mensagem}`).
CV03	BRL em centavos inteiros	Todo valor monetário trafega e persiste como inteiro (`precoCentavos`), evitando erro de ponto flutuante.
CV04	Datas	ISO-8601 UTC nos contratos; formato local na interface.
CV05	Termos financeiros	Indicadores explicados na interface; simulações rotuladas como premissas.
CV06	Simulações rastreáveis	Entradas, premissas e data de referência acompanham todo resultado.
CV07	C4 consistente	Diagramas indicam tipo, responsabilidade, tecnologia e relações rotuladas.
CV08	Eventos nomeados e versionados (nova)	`investlab.<contexto>.<evento>` com campo `versao` ; esquema em `contracts/events/*.vN.json` .
CV09	Estrutura de pastas por camada e por fatia (nova)	`src/domain` , `src/application/ports` , `src/features/<caso-de-uso>` , `src/infrastructure` , `src/api` ; verificada por `test/arquitetura.test.ts` .

3. Contexto e Escopo

O InvestLab é delimitado como uma plataforma analítica. O investidor fornece perfil e parâmetros; fontes externas fornecem dados financeiros e econômicos; o sistema processa essas informações e devolve indicadores, simulações e comparações. A execução de investimentos permanece fora da fronteira do sistema.

3.1 Contexto de negócio

No contexto de negócio, o sistema é visto como uma única caixa. O investidor é o ator principal. B3, CVM, Banco Central, Tesouro Direto e FGC aparecem como sistemas externos de onde o InvestLab obtém dados. A decomposição interna em microfrontends, gateway, BFF, microserviços e Functions não altera essa fronteira.

3.1.1 C4 Nível 1 - Diagrama de Contexto

Figura 1 - C4 Nível 1: contexto do sistema InvestLab

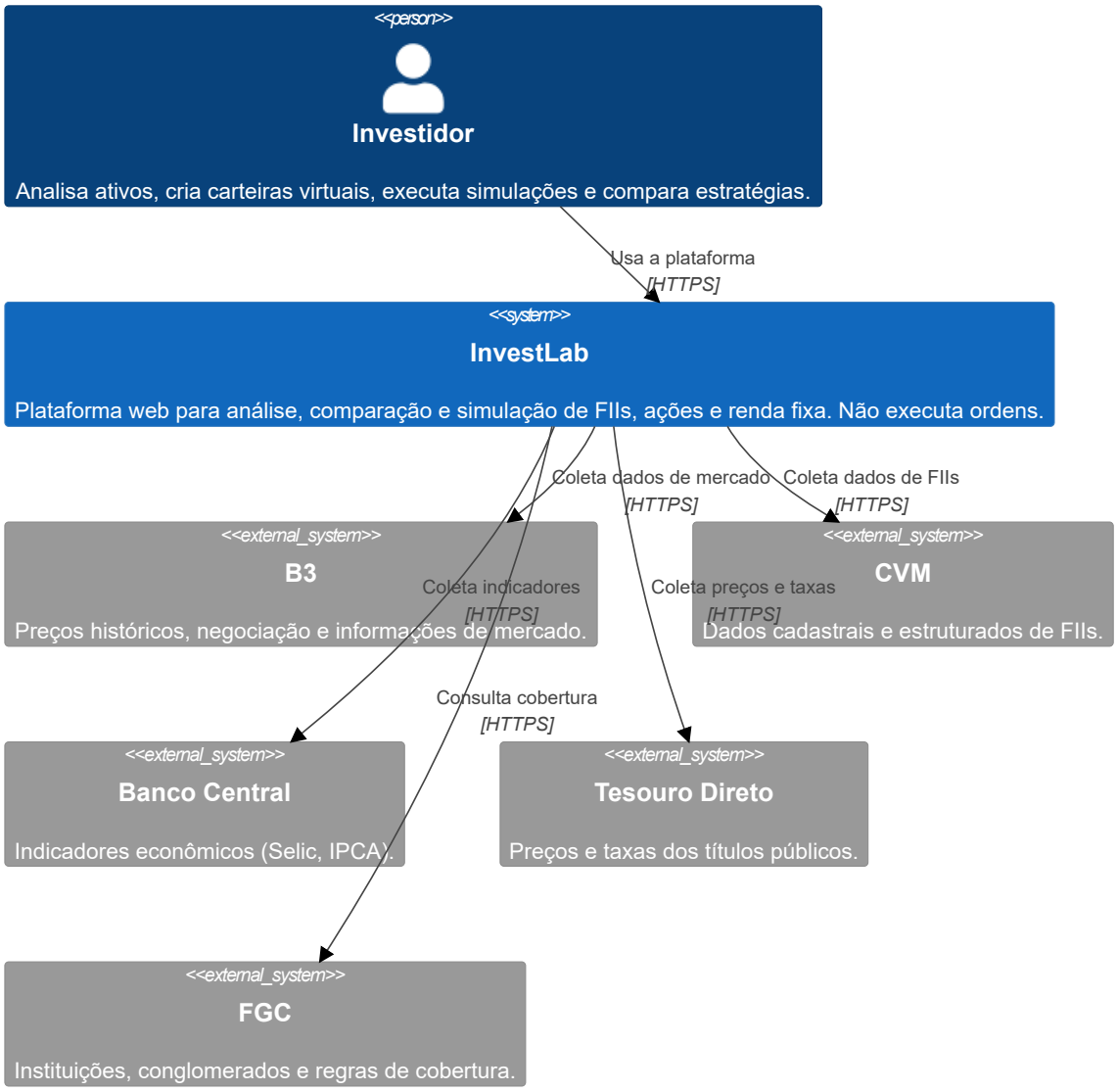


Figura 1 - C4 Nível 1: contexto do sistema. Legenda C4: *Person* representa o usuário; *Software System* representa o InvestLab; *External System* (cinza) representa sistemas fora da responsabilidade do projeto. As relações indicam o sentido da interação.

3.2 Interfaces de negócio

Parceiro	Entradas recebidas pelo InvestLab	Saídas do InvestLab
Investidor	Cadastro, perfil, ativos, ofertas, carteira, aportes e premissas.	Indicadores, gráficos, simulações, backtests, comparações, alertas e dashboard.

B3	Dados de preços e ativos.	Solicitações de consulta ou coleta.
CVM	Dados públicos e cadastrais de FIIs.	Parâmetros de consulta.
Banco Central	Selic e séries selecionadas.	Código da série e período.
Tesouro Direto	Títulos, taxas, preços e vencimentos.	Parâmetros de consulta.
FGC	Instituições e regras de cobertura.	Identificação de instituição ou conglomerado.

3.3 Contexto técnico

O contexto técnico conecta o navegador ao Azure Static Web Apps (microfrontends), que fala exclusivamente com o API Gateway (AWS). O gateway encaminha ao BFF, hospedado no Azure, que orquestra os microserviços, a Azure Function e, indiretamente, o Azure Service Bus. Os bancos ficam em serviços gerenciados (MongoDB Atlas e Azure SQL). O detalhamento dos containers está na seção 5 e a implantação física na seção 7.

Canal	Protocolo / tecnologia	Observação
Navegador → mfe-shell / remotes	HTTPS, ES modules (Module Federation)	Remotes carregados em runtime por `remoteEntry.js`.
mfe-* → API Gateway	HTTPS / JSON (REST)	Única URL pública de API; CORS restrito ao domínio do shell.
API Gateway → BFF	HTTP proxy	Roteamento `/aggregated-data` e `/api/*`.
BFF → ms-ativos / ms-carteira / fn-simulacao	HTTP / JSON, timeout 5 s	Chamadas em paralelo; falha parcial degrada graciosamente.
ms-ativos → Service Bus → fn-simulacao → ms-carteira	AMQP (tópico + assinatura) e HTTP	Fluxo assíncrono orientado a eventos (seção 6.1).
ms-ativos → MongoDB Atlas; ms-carteira → Azure SQL	MongoDB wire protocol (TLS); TDS (TLS)	Credenciais via variáveis de ambiente.
ms-ativos → fontes externas	HTTPS (APIs/CSV)	Coleta periódica (Timer trigger na Entrega 3).

4. Estratégia de Solução

A estratégia de solução da versão 2.0 decompõe o InvestLab por **capacidades de negócio** (ativos, carteira, simulação) e aplica os estilos arquiteturais estudados na disciplina de forma *intencional*: cada estilo resolve um problema concreto do sistema, e o custo adicional de cada um é registrado nas consequências dos ADRs (seção 9).

4.1 Princípios arquiteturais

- **Entrada única e frontend desacoplado:** o microfrontend só conhece o API Gateway; o gateway só conhece o BFF.
- **Um BFF por tipo de cliente:** o `bff-web` adapta e agrega dados para a web (`GET /aggregated-data`); não possui banco nem regra de negócio de domínio.
- **Microserviços donos dos seus dados:** `ms-ativos` (MongoDB) e `ms-carteira` (Azure SQL) não compartilham base nem esquema; comunicam-se por eventos.
- **Serverless para cálculo sob demanda e reação a eventos:** a simulação (RF07/RF08) e o consumo de eventos rodam em Azure Functions, pagando apenas pelo uso.
- **Consistência eventual explícita:** a cotação muda no ms-ativos e as carteiras são reavaliadas de forma assíncrona e idempotente; o usuário vê `dataReferencia` .
- **Clean Architecture + Vertical Slice em cada container:** domínio sem dependências, portas em `application` , cada caso de uso em uma fatia com endpoint, handler e teste; infraestrutura nas bordas.
- **Arquitetura verificável:** as regras acima são testes automatizados (`test/arquitetura.test.ts`), não apenas convenção.
- **Reprodutibilidade:** premissas, data de referência e centavos inteiros em todo contrato (ADR-06 mantida).

4.2 Estilos arquiteturais adotados e onde aparecem

Estilo / padrão	Problema que resolve no InvestLab	Onde está no código
Microfrontend	Equipes evoluírem "Ativos" e "Carteira" sem redeploy do shell; falha de um módulo não derruba a aplicação.	`frontend/mfe-shell` (host), `frontend/mfe-ativos` , `frontend/mfe-carteira` (remotes, `@originjs/vite-plugin-federation`)
API Gateway	Entrada única com CORS, throttling e autenticação centralizados; serviços nunca expostos.	`gateway/openapi-gateway.yaml` (AWS HTTP API, integração `http_proxy`)
Backend for Frontend	Reduzir chamadas e payload da web; agregar três fontes em um JSON.	`services/bff-web` - `features/aggregated-data` , proxies
Microservices	Escalar e implantar ativos e carteira independentemente; tecnologia de banco adequada a cada domínio.	`services/ms-ativos` , `services/ms-carteira`
Database per Service	Isolar esquemas; evitar acoplamento por banco.	MongoDB Atlas (`MongoAtivoRepository`) e Azure SQL (`SqlCarteiraRepository` , `sql/schema.sql`)
Serverless	Cálculo elástico da simulação; consumidor de eventos sem servidor dedicado.	`services/fn-simulacao` - `functions/simular.ts` (HTTP), `functions/cotacaoAtualizada.ts` (Service Bus)

Arquitetura Orientada a Eventos	Propagar mudança de cotação para carteiras sem acoplamento síncrono entre microserviços.	Evento `investlab.ativos.cotacao-atualizada`; `ServiceBusEventPublisher`; `contracts/events`
Clean Architecture	Proteger regras financeiras de frameworks, bancos e HTTP; testabilidade.	`src/domain`, `src/application/ports`, `src/infrastructure`, `src/api` em cada serviço
Vertical Slice	Mudança localizada por caso de uso; cada fatia carrega endpoint, handler e teste.	`src/features/<caso-de-uso>/{endpoint.ts, handler.ts, handler.test.ts}`
Testes de arquitetura	Impedir que a regra das dependências se erosione.	`test/arquitetura.test.ts` (tsarch) em todos os serviços

4.3 Estratégia de dados

Cada microserviço possui o modelo de dados que melhor serve ao seu domínio: ativos são documentos (um ticker, um documento, consultas por tipo), enquanto a carteira é relacional (carteira, posições e transações com integridade referencial e cálculo de patrimônio). A integração entre eles é exclusivamente por evento: o ms-ativos publica a nova cotação; a Function consome e aciona a fatia `atualizar-precos` do ms-carteira, que é idempotente. A última base válida nunca é substituída por dado mais antigo (regra de domínio em `Ativo.atualizarCotacao`), e o usuário sempre vê a data de atualização.

4.4 Software Architecture Canvas

1. Problema / Necessidade

- Dados de investimento fragmentados
- Comparações superficiais (só rentabilidade bruta)
- Dificuldade de simular longo prazo
- Falta de ambiente único de análise

2. Usuários / Stakeholders

- Investidor
- Equipe de desenvolvimento
- Professor / avaliadores
- Provedores de dados
- Responsável pela manutenção

3. Valor / Resultado

- Catálogo de ativos com cotação atual
- Carteira virtual reavaliada automaticamente
- Simulação com 3 cenários
- Dashboard agregado em um único JSON
- Sem recomendação automática

4. Qualidades-chave

- Correção / precisão (centavos, determinismo)
- Usabilidade
- Desempenho (agregação paralela)
- Confiabilidade dos dados
- Segurança; modificabilidade; resiliência

5. Estilo e princípios arquiteturais

- Microfrontend (shell + remotes) → API Gateway → BFF → Microservices com Database per Service
- Serverless (Azure Functions) para simulação e consumo de eventos
- Arquitetura Orientada a Eventos (Service Bus, consistência eventual explícita)
- Clean Architecture + Vertical Slice em cada serviço; regra das dependências verificada por testes
- Integrações externas isoladas; resultados reproduzíveis e datados; REST + JSON sobre HTTPS; eventos versionados

6. Blocos principais

- mfe-shell, mfe-ativos, mfe-carteira (React)
- API Gateway (AWS) · bff-web (Node)
- ms-ativos (Node + MongoDB)
- ms-carteira (Node + Azure SQL)
- fn-simulacao (Azure Functions)
- Azure Service Bus

7. Integrações

- B3 - mercado
- CVM - dados públicos de FIIs
- Banco Central - indicadores
- Tesouro Direto - títulos
- FGC - instituições / cobertura
- Evento `cotacao-atualizada` entre serviços

8. Dados

- Ativos, preços, proventos (MongoDB)
- Carteiras, posições, transações (Azure SQL)
- Perfis e simulações (Azure SQL, evolução)
- Ofertas de renda fixa e instituições / FGC (MongoDB, evolução)
- Premissas e data de referência em todo resultado

9. Operação / Observabilidade

- Logs JSON estruturados; Application Insights
- Health checks; métricas de duração
- Dead-letter monitorada; reentrega
- Última base válida preservada
- CI: testes unitários + de arquitetura; imagens no Docker Hub

10. Limites / Não-objetivos

- Sem compra e venda; sem custódia
- Sem Open Finance na V2
- Sem cotação tick-by-tick
- Sem apuração fiscal completa
- Sem recomendação automática por IA

11. Riscos principais

- Consistência eventual confundir o usuário
- Cold start e limites dos planos gratuitos
- Multi-cloud (Azure + AWS)
- Erros em fórmulas financeiras
- Depuração distribuída

12. Decisões arquiteturais centrais

Microfrontend + API Gateway + BFF + 2 microserviços (ativos/MongoDB, carteira/Azure SQL) + Azure Functions + Service Bus; TypeScript/Node em todos os containers; Clean Architecture + Vertical Slice com testes de arquitetura (tsarch) no CI; cálculos determinísticos em centavos; segredos só por ambiente; Azure como nuvem principal e AWS para o gateway (ADR-08 a ADR-15; ADR-01 a ADR-04 da Entrega 1 substituídas). Canvas elaborado para consolidar as decisões desta documentação; não substitui as seções detalhadas do arc42.

Figura 2 - Software Architecture Canvas do InvestLab, no mesmo formato de 12 blocos da Entrega 1 (verde: objetivos; amarelo: solução; azul: avaliação). Mapeamento Canvas × arc42 conforme a aula: 1 e 3 → 1.1; 4 → 1.2; 2 e 7 → 3.1; 5 e 6 → 4 e 5; 8 → 5.6; 9 → 8; 10 → 1.1.3; 11 → 11; 12 → 9.

5. Visão de Blocos de Construção

A visão de blocos refina a solução em níveis de abstração do C4: sistema (seção 3), containers (5.1), componentes (5.2 e 5.3) e código em UML (5.4 e 5.5), além do DER (5.6). As cores seguem a notação C4:

Person System Container Component External

5.1 C4 Nível 2 - Diagrama de Containers

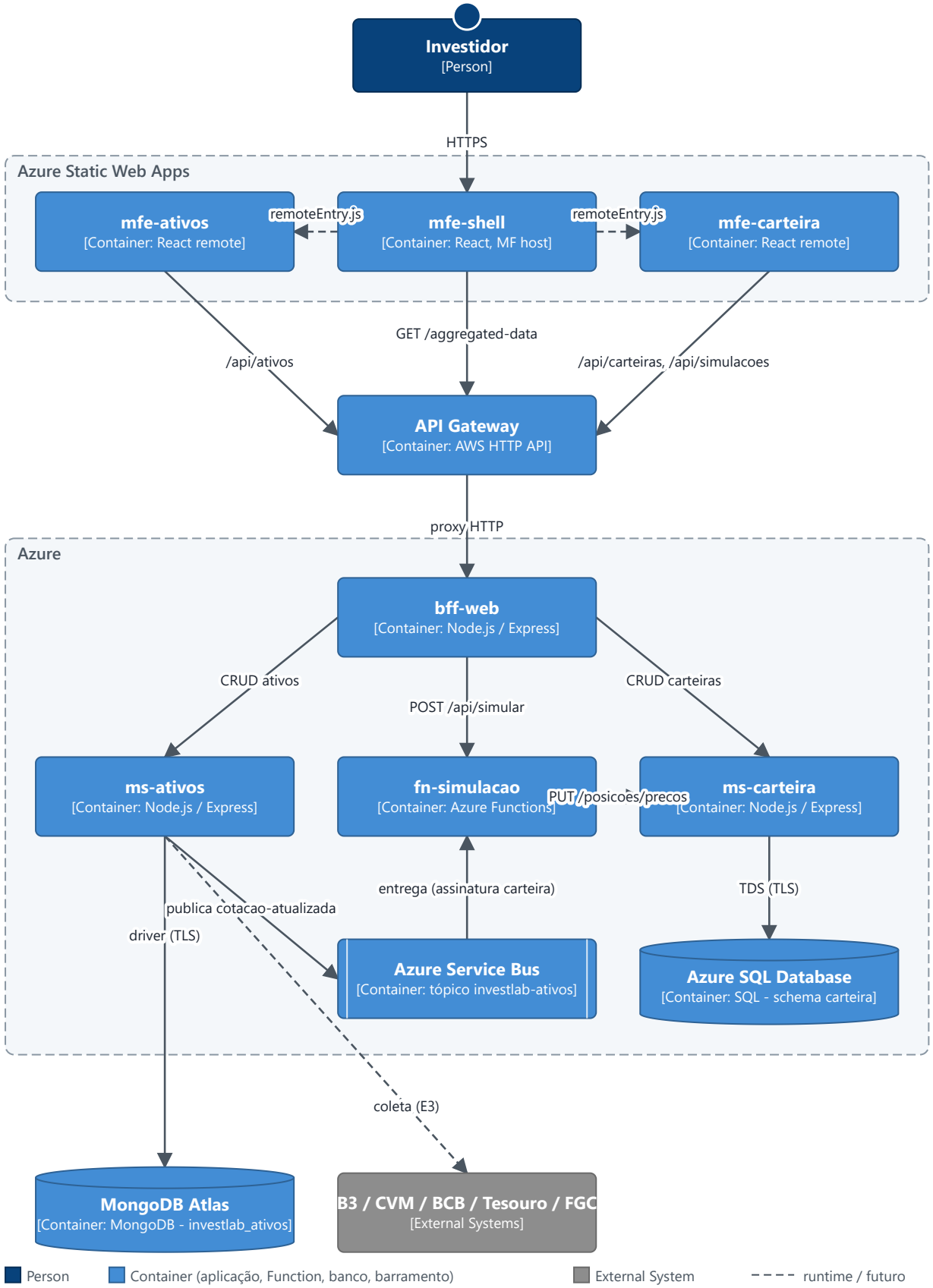


Figura 3 - C4 Nível 2: containers do InvestLab. Setas contínuas: chamadas síncronas; tracejadas: carregamento em runtime ou integração futura. O único caminho público é Investidor → Static Web Apps → API Gateway. Tecnologias e responsabilidades detalhadas na tabela 5.1.1.

5.1.1 Responsabilidades dos containers

Container	Tecnologia	Responsabilidade	Estilo
mfe-shell	React 18, Vite, `@originjs/vite-plugin-federation` (host), Azure SWA	Layout, navegação, dashboard (consome `/aggregated-data`), carregamento dos remotes com *error boundary*.	Microfrontend
mfe-ativos	React remote (`./App`)	Listar/filtrar/cadastrar ativos (RF03, RF04, RF09) via BFF.	Microfrontend
mfe-carteira	React remote (`./App`)	Criar carteira, registrar aporte, ver posições (RF15) e simular (RF07, RF08) via BFF.	Microfrontend
API Gateway	AWS API Gateway HTTP API (OpenAPI importado)	Entrada única; CORS; throttling; autenticação JWT (Entrega 3); proxy para o BFF.	API Gateway
bff-web	Node.js 22, Express 5, swagger-ui	`GET /aggregated-data` (agrega ativos + carteiras + simulação em paralelo); proxies `/api/ativos`, `/api/carteiras`, `/api/simulacoes`; degradação graciosa.	BFF
ms-ativos	Node.js 22, Express 5, driver MongoDB, SDK Service Bus	CRUD de ativos, atualização de cotação com regra de data de referência, publicação do evento.	Microservice, Database per Service, EDA (produtor)
ms-carteira	Node.js 22, Express 5, mssql	Carteiras, posições com preço médio ponderado, transações, patrimônio; endpoint idempotente de reavaliação de preços.	Microservice, Database per Service, EDA (consumidor final)
fn-simulacao	Azure Functions v4 (Node), plano Consumption	`POST /api/simular` (RF07/RF08); `ServiceBusTopicTrigger` que repassa a cotação ao ms-carteira.	Serverless, EDA (consumidor)
Azure Service Bus	Tópico `investlab-ativos`, assinatura `carteira`, dead-letter	Desacoplar produtor e consumidores; reentrega; DLQ.	EDA
MongoDB Atlas	M0 (free), base `investlab_ativos`, coleção `ativos`	Persistência exclusiva do ms-ativos.	Database per Service
Azure SQL Database	Free (1 DTU), schema `carteira`	Persistência exclusiva do ms-carteira.	Database per Service

5.2 C4 Nível 3 - Componentes do ms-ativos

O diagrama mostra a estrutura interna comum a todos os serviços Node (ms-ativos, ms-carteira e, com menos fatias, bff-web): o *composition root* registra cada fatia vertical; as fatias usam o domínio e as portas; a infraestrutura implementa as portas. As setas de dependência apontam para dentro (Clean Architecture).

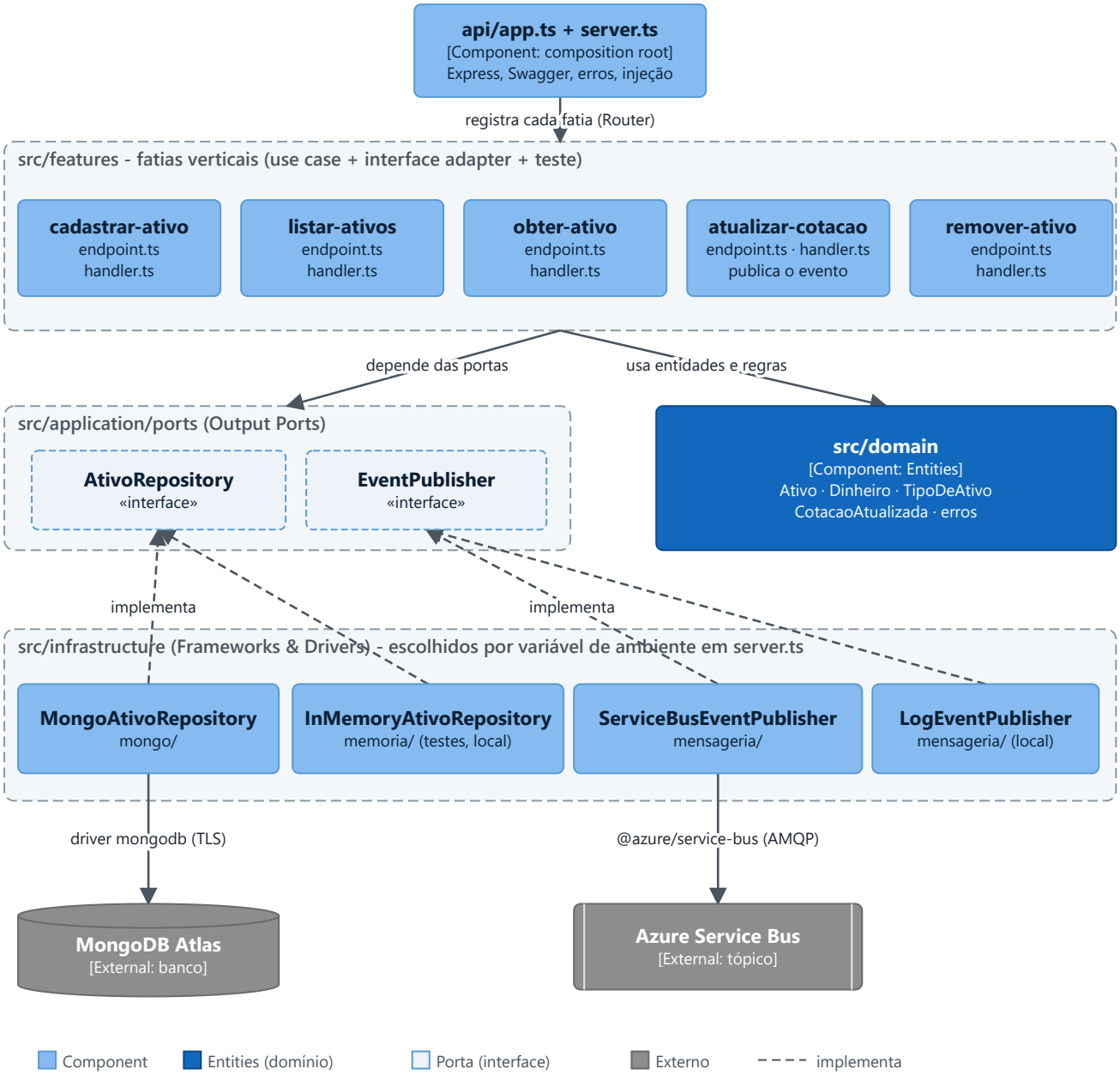


Figura 4 - C4 Nível 3: componentes do ms-ativos. As fatias dependem apenas de `domain` e das portas; `infrastructure` só implementa portas; `api` é o único lugar que conhece todos.

5.2.1 Componentes e responsabilidades (ms-ativos e ms-carteira)

Componente	Responsabilidade	Equivalente no ms-carteira
`domain/Ativo`, `Dinheiro`	Entidade e value object com invariantes (ticker válido, preço > 0, data de referência não retrocede); gera o evento `CotacaoAtualizada`.	`domain/Carteira` (agregado), `Posicao` (preço médio ponderado), `Transacao`
`application/ports/AtivoRepository`	Porta de saída de persistência.	`CarteiraRepository` (inclui `listarComAtivo` para o consumidor de eventos)
`application/ports/EventPublisher`	Porta de saída para o barramento.	- (o ms-carteira é consumidor, via endpoint)

`features/cadastrar-ativo`	POST /ativos; valida com zod, cria entidade, rejeita duplicado (409).	`criar-carteira` (POST /carteiras)
`features/listar-ativos`, `obter-ativo`	GET /ativos?tipo=, GET /ativos/{ticker}.	`listar-carteiras`, `obter-carteira`
`features/atualizar-cotacao`	PUT /ativos/{ticker}/cotacao; persiste e publica o evento (EDA, produtor).	`registrar-aporte` (POST /carteiras/{id}/aportes) e `atualizar-precos` (PUT /posicoes/precos, idempotente - EDA, consumidor)
`features/remover-ativo`	DELETE /ativos/{ticker}.	`remover-carteira`
`infrastructure/mongo`, `mensageria`, `memoria`	Adaptadores: MongoDB Atlas, Service Bus, memória (testes/local), log.	`infrastructure/sql/SqlCarteiraRepository` (mssql, transação), `memoria`
`api/app.ts`, `server.ts`, `openapi.ts`, `erros.ts`	Composition root, seleção de adaptadores por ambiente, Swagger UI em `/docs`, mapeamento de erros → HTTP.	idem
`test/arquitetura.test.ts`	7 regras de dependência (seção 8.2).	7 regras

5.3 C4 Nível 3 - Componentes do bff-web e da fn-simulacao

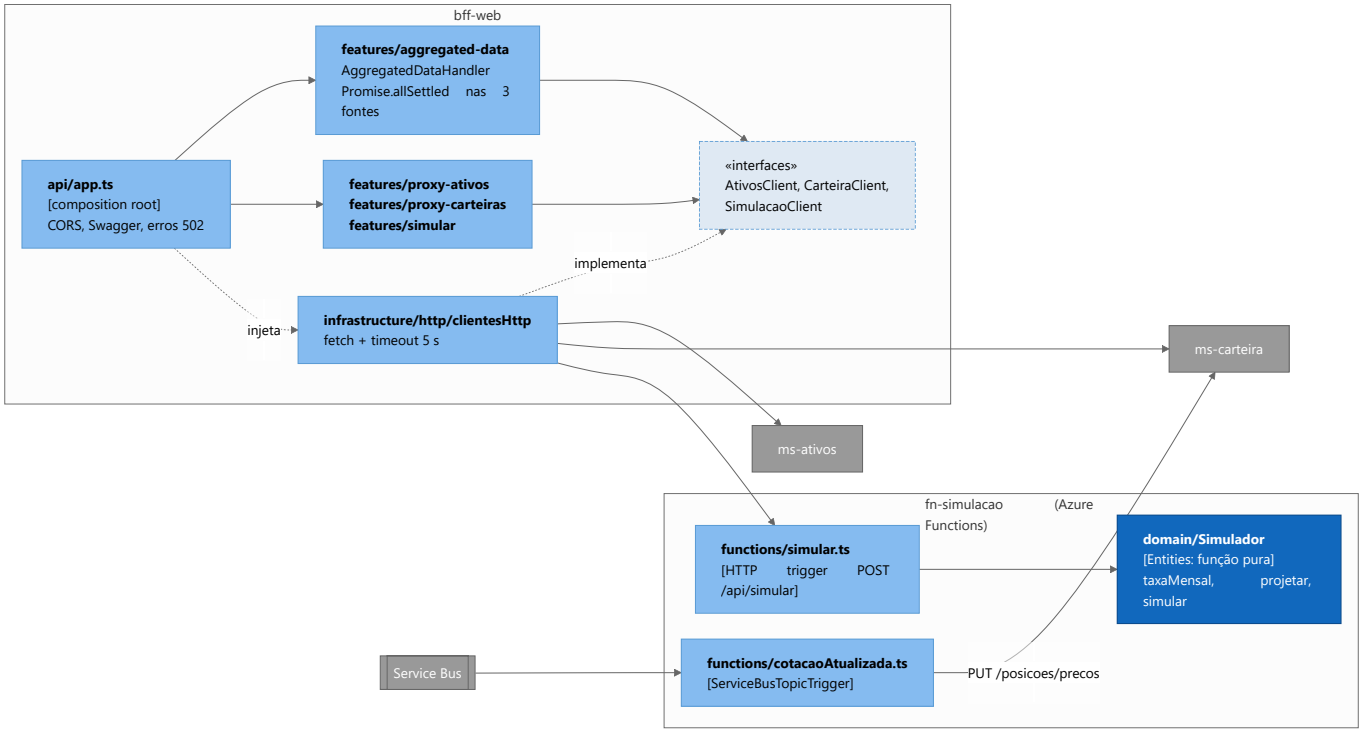


Figura 5 - C4 Nível 3: componentes do bff-web (portas para os upstreams) e da fn-simulacao (domínio puro isolado do SDK do Azure).

5.4 C4 Nível 4 - UML de componentes

O diagrama de componentes UML detalha os contratos (interfaces) entre as fatias e os adaptadores do ms-ativos, evidenciando a **inversão de dependência**: casos de uso dependem de interfaces; adaptadores as realizam.

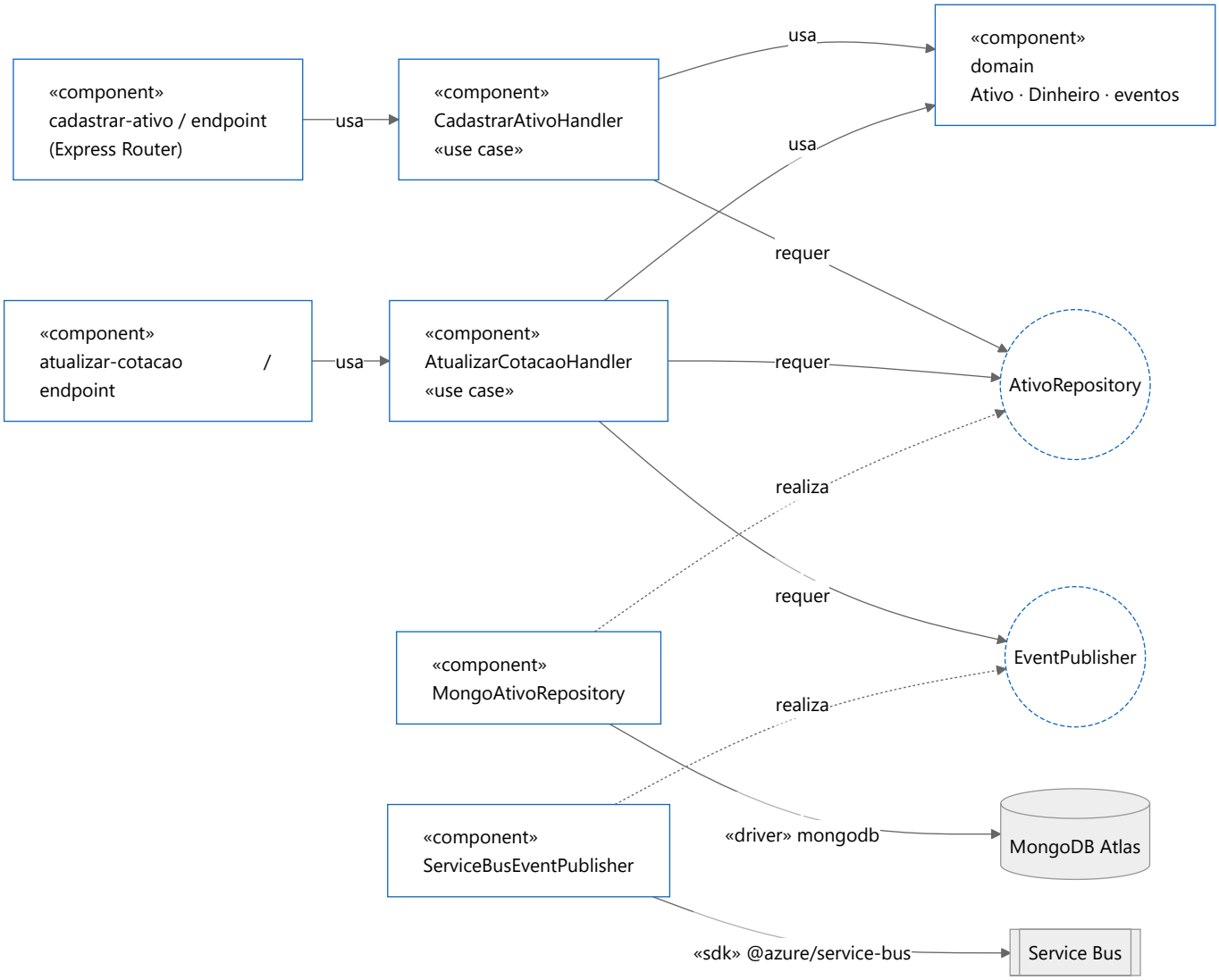


Figura 6 - UML: diagrama de componentes do ms-ativos (interfaces como lollipops; linhas tracejadas = realização). Mesmo padrão em ms-carteira (CarteiraRepository ← SqlCarteiraRepository) e bff-web (AtivosClient/CarteiraClient/SimulacaoClient ← clientes HTTP).

5.5 C4 Nível 4 - UML de classes

As classes representam o núcleo do domínio de cada microserviço e da Function, exatamente como implementadas em `src/domain`. Não há dependência entre os domínios dos dois microserviços; o acoplamento é apenas pelo contrato do evento (DTO `CotacaoAtualizada`).

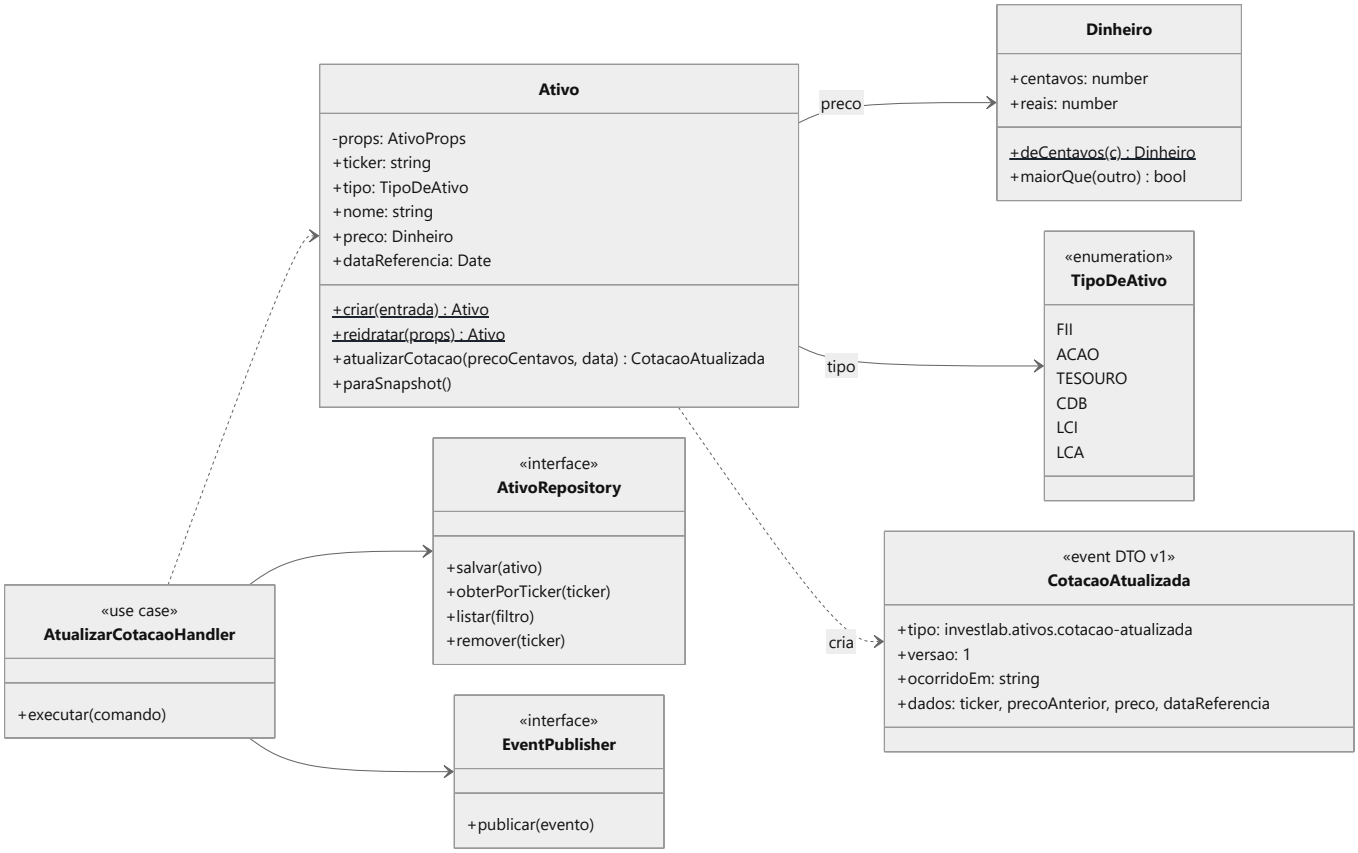


Figura 7 - UML: diagrama de classes do domínio do ms-ativos (`src/domain`) e das portas usadas pelo caso de uso de atualização de cotação.

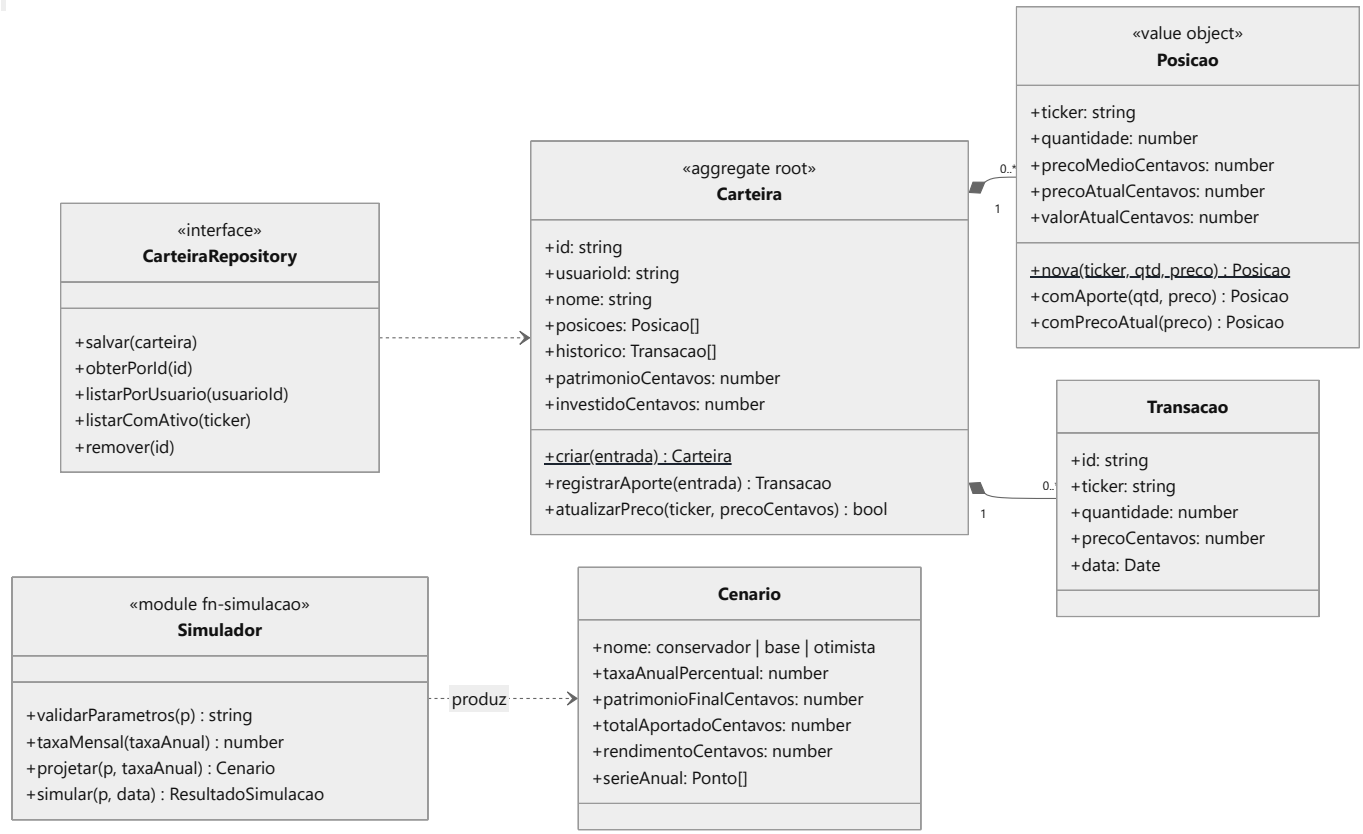


Figura 8 - UML: diagrama de classes dos domínios do ms-carteira e da fn-simulacao. Não há dependência de código entre os domínios: o acoplamento com o ms-ativos é apenas pelo contrato do evento `CotacaoAtualizada`, consumido de forma assíncrona (ver Figura 11).

5.6 Diagrama de Entidades e Relacionamentos

Com Database per Service há dois modelos físicos independentes, mostrados separadamente. O ms-ativos armazena documentos no MongoDB Atlas (base `investlab_ativos`); o ms-carteira usa o schema relacional `carteira` no Azure SQL (`sql/schema.sql`). As entidades marcadas como **implementada** existem no código desta entrega; as marcadas como **prevista** vêm do modelo de dados da Entrega 1 e estão posicionadas na base do serviço que será dono delas (seção 11.1). **Não existe chave estrangeira entre as bases**: `POSICAO.Ticker` e `HISTORICO_PRECO.ticker` referenciam um ativo apenas logicamente, e a consistência entre serviços é mantida pelo evento de cotação.

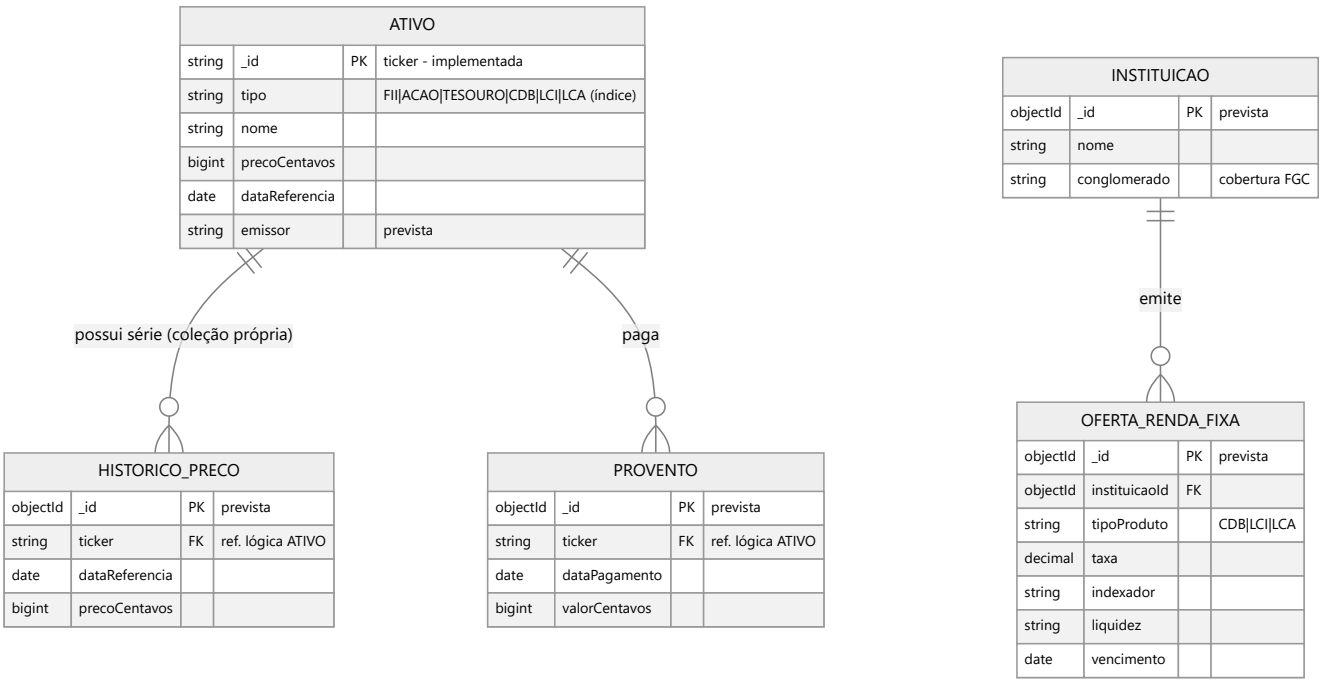


Figura 9 - DER da base do ms-ativos (MongoDB Atlas, `investlab_ativos`): coleção `ativos` implementada; `historico_precos`, `proventos`, `instituicoes` e `ofertas_renda_fixa` previstas (RF04, RF09-RF13), herdadas do modelo da Entrega 1.

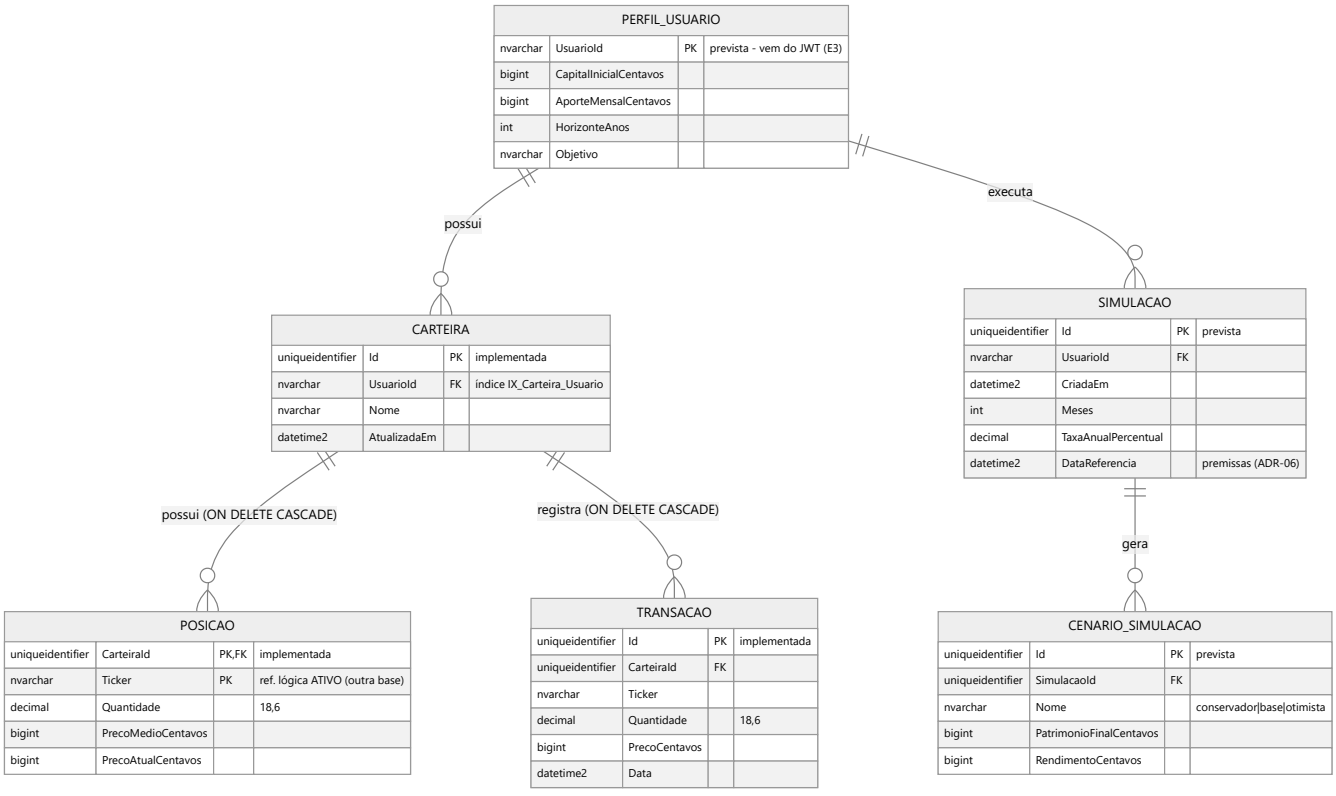


Figura 10 - DER da base do ms-carteira (Azure SQL, schema `carteira`): `Carteira`, `Posicao` e `Transacao` implementadas (`sql/schema.sql`); `PerfilUsuario`, `Simulacao` e `CenarioSimulacao` previstas (RF02, RF07, RF08) para persistir premissas e resultados das simulações que hoje a fn-simulacao calcula sem armazenar.

5.7 Rastreabilidade entre níveis

Vista	Foco	Localização
C4 Nível 1	InvestLab + investidor + sistemas externos	Seção 3.1.1 (Fig. 1)
C4 Nível 2	Microfrontends, gateway, BFF, microserviços, Function, barramento, bancos	Seção 5.1 (Fig. 3)
C4 Nível 3	Componentes do ms-ativos; do bff-web e da fn-simulacao	Seções 5.2 e 5.3 (Figs. 4 e 5)
C4 Nível 4 - Componentes UML	Interfaces (portas) e realizações (adaptadores)	Seção 5.4 (Fig. 6)
C4 Nível 4 - Classes UML	Domínios dos serviços e da Function	Seção 5.5 (Figs. 7 e 8)
C4 Nível 4 - Sequência UML	Fluxo de eventos (cotação) e agregação (dashboard)	Seção 6 (Figs. 11 e 12)
DER	Persistência por serviço	Seção 5.6 (Figs. 9 e 10)
Implantação	Azure, AWS, Atlas, Docker Hub, GitHub	Seção 7 (Fig. 13)
Clean Architecture / Vertical Slice	Regra das dependências e estrutura de pastas	Seção 8.1 (Fig. 14)

6. Visão de Runtime

A visão de runtime descreve como os blocos colaboram em cenários relevantes. Os dois cenários principais exercitam, respectivamente, a **arquitetura orientada a eventos** (atualização de cotação propagada para as carteiras) e o **BFF** (agregação do dashboard). Ambos foram executados ponta a ponta no ambiente local durante a construção desta entrega.

6.1 Cenário: atualizar cotação e reavaliar carteiras (EDA)

- Um operador (ou o job de coleta da Entrega 3) envia a nova cotação pelo gateway.
- O BFF encaminha ao ms-ativos; a entidade valida que a data de referência não retrocede.
- O ms-ativos persiste no MongoDB e publica `cotacao-atualizada` no tópico.
- A Function com `ServiceBusTopicTrigger` consome e chama `PUT /posicoes/precos` no ms-carteira.
- O ms-carteira atualiza as posições afetadas (idempotente) e persiste no Azure SQL.
- Em caso de falha, o Service Bus reentrega; após as tentativas, a mensagem vai para a dead-letter.

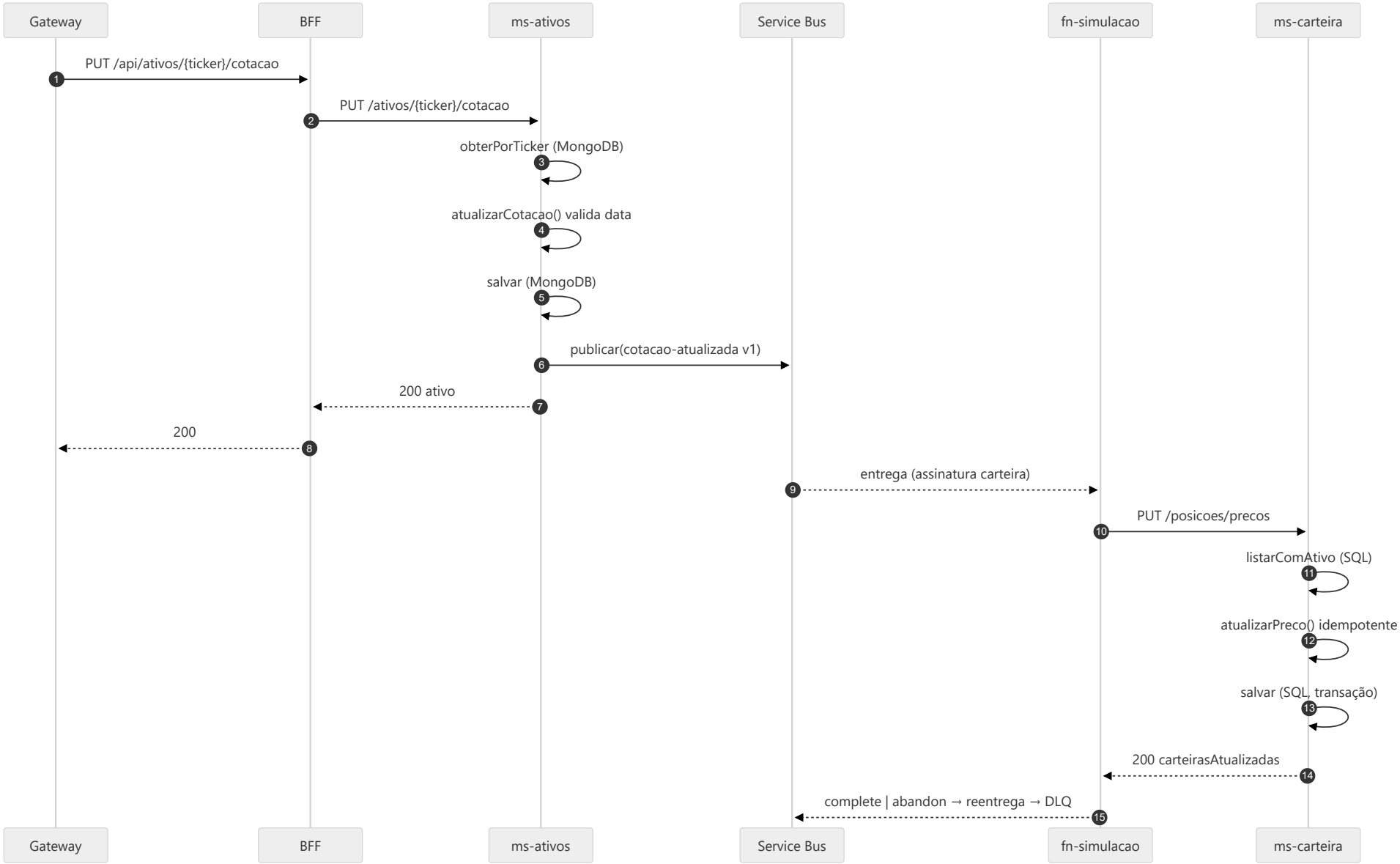


Figura 11 - UML: diagrama de sequência da atualização de cotação. Os passos 1-8 são síncronos; 9-15 são assíncronos (consistência eventual). A persistência em MongoDB e Azure SQL está indicada como auto-mensagens.

6.2 Cenário: carregar o dashboard agregado (BFF)

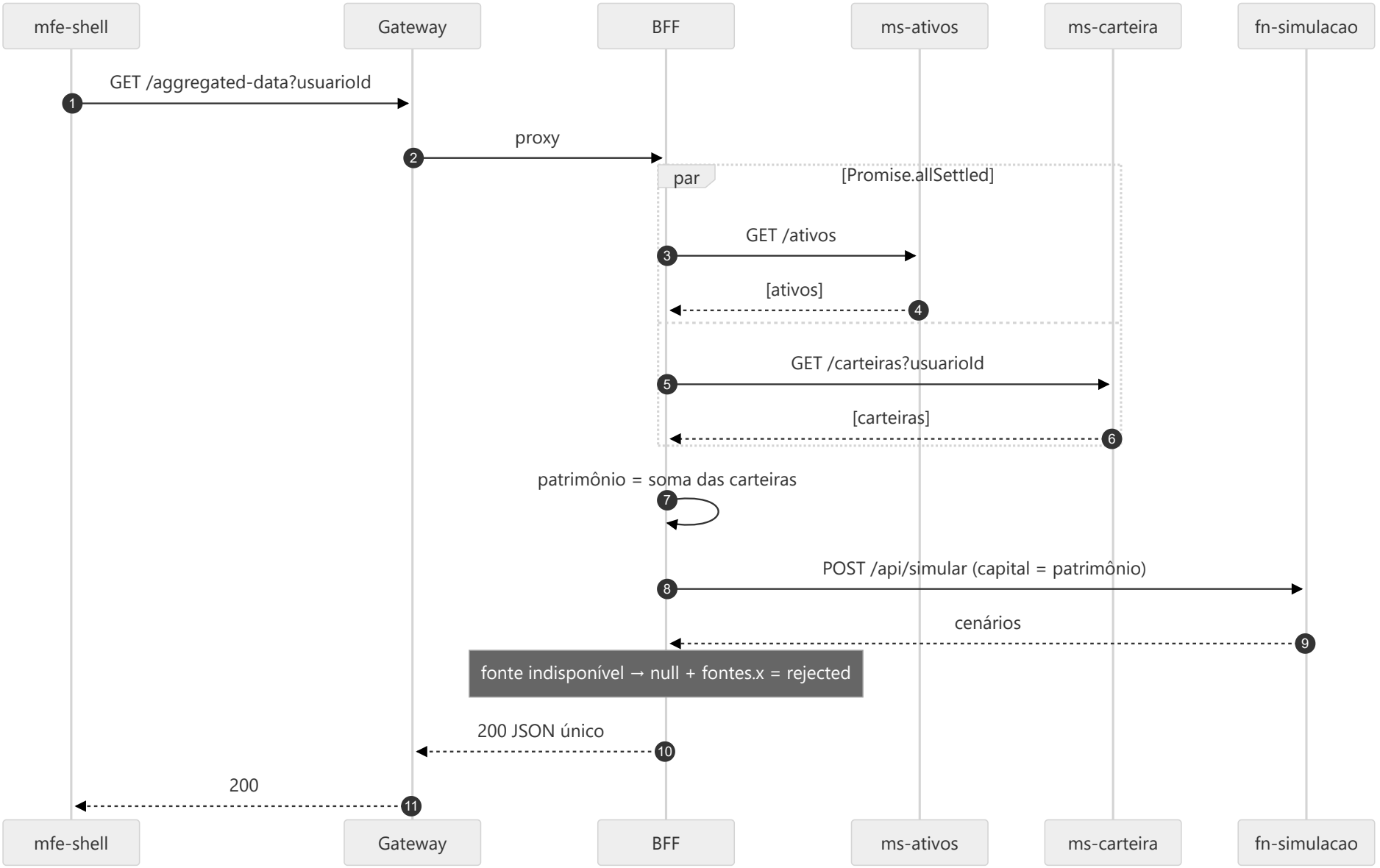


Figura 12 - UML: diagrama de sequência do `GET /aggregated-data`. O BFF reduz três chamadas do cliente a uma e degrada graciosamente.

6.3 Outros runtimes relevantes

Cenário	Cadeia principal	Resultado
Cadastrar ativo	mfe-ativos → Gateway → BFF (proxy) → ms-ativos `cadastrar-ativo` → MongoDB	201 com `Location`; 409 se ticker existe; 422 se regra de domínio falha.
Registrar aporte	mfe-carteira → Gateway → BFF → ms-carteira `registrar-aporte` → Azure SQL (transação)	Posição com preço médio recalculado; transação guardada para rastreabilidade (Q07).
Simular patrimônio	mfe-carteira → Gateway → BFF `/api/simulacoes` → fn-simulacao HTTP → `Simulador.simular`	Três cenários em centavos inteiros, determinísticos (Q01).
Carregar remote	mfe-shell → `remoteEntry.js` do remote (SWA)	Falha do remote é contida por *error boundary*; o shell continua.
Fonte externa indisponível	Coleta falha → `atualizar-cotacao` não é chamada	Última base válida permanece; `dataReferencia` informa ao usuário (Q05).
Mensagem inválida no tópico	Trigger → `zod.parse` lança → abandon → reentregas → dead-letter	Mensagem isolada para análise; demais mensagens seguem.
Inicialização de um serviço	`server.ts` lê variáveis de ambiente → escolhe adaptadores (Mongo/SQL/Service Bus ou memória/log) → registra fatias → Swagger	Mesmo binário roda local e em nuvem; nenhum segredo no código.

7. Visão de Implantação

A implantação usa os serviços gratuitos disponíveis à equipe (Azure for Students, MongoDB Atlas M0, AWS Free Tier, Docker Hub e GitHub). O gateway fica na AWS por exigência do enunciado da Entrega 3; o restante fica no Azure, onde já foram exercitados Functions, Static Web Apps e Service Bus nas atividades formativas.

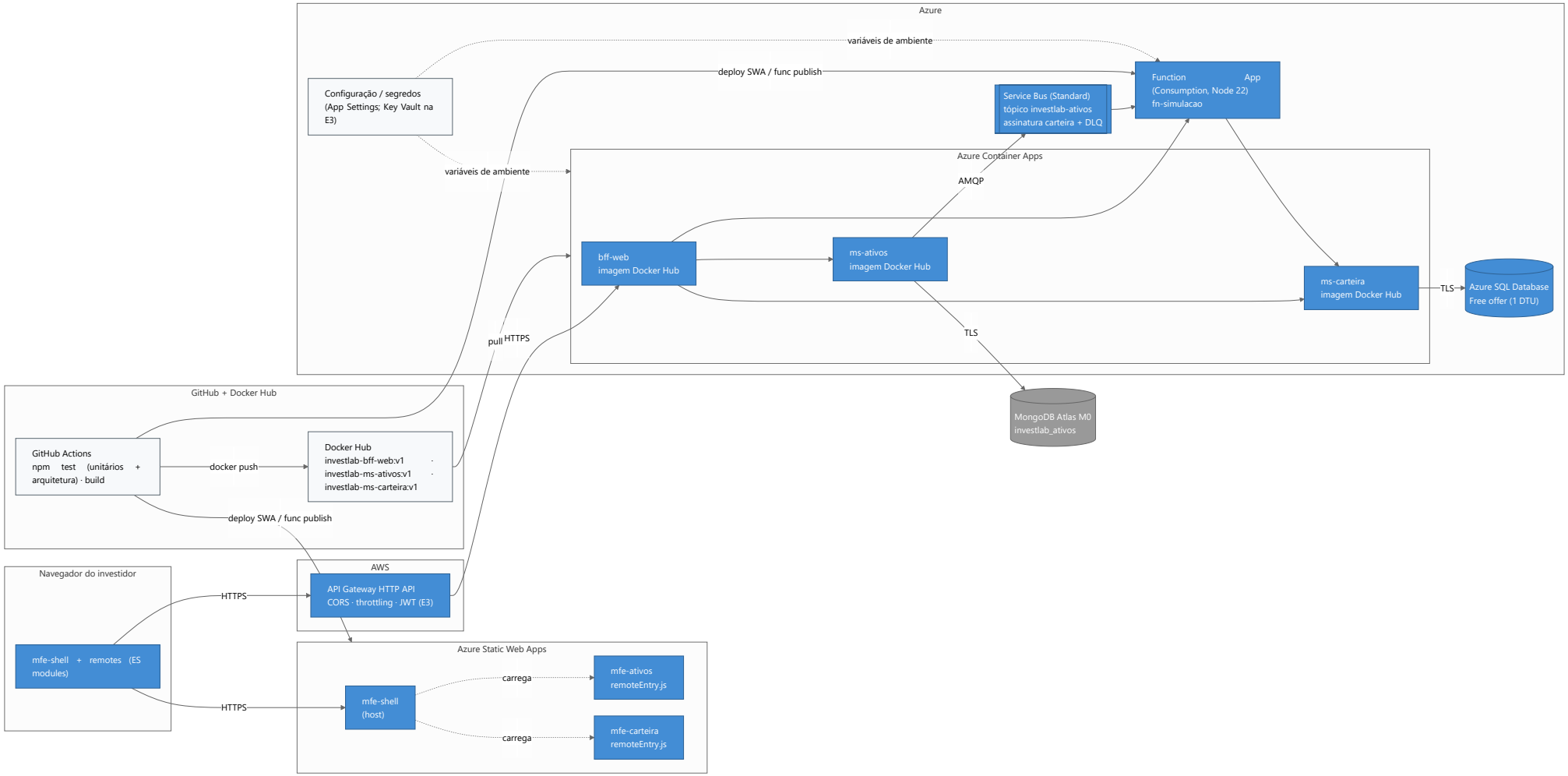


Figura 13 - Visão de implantação. Os três microfrontends são aplicações separadas no Static Web Apps; os três serviços Node rodam como containers a partir das imagens publicadas no Docker Hub; a Function roda no plano Consumption.

7.1 Elementos de infraestrutura

Elemento	Serviço / plano	O que executa	Limites relevantes
Static Web Apps	Azure, Free	mfe-shell, mfe-ativos, mfe-carteira (build Vite, deploy por GitHub Actions)	100 GB/mês de banda; sem SLA
API Gateway	AWS HTTP API, Free Tier	Rotas `/aggregated-data`, `/api/{proxy+}`, `/health` → BFF	1 M req/mês grátis (12 meses)
Container Apps	Azure, Consumption	bff-web (:3000), ms-ativos (:3001), ms-carteira (:3002) - imagens do Docker Hub	Scale-to-zero; cold start ~segundos
Function App	Azure Functions, Consumption, Node 22	fn-simulacao: HTTP `simular` e trigger `cotacaoAtualizada`	1 M execuções/mês; cold start
Service Bus	Azure, Standard (tópicos exigem Standard)	Tópico `investlab-ativos`, assinatura `carteira`, dead-letter	Custo base mensal baixo; 12,5 M operações
Azure SQL Database	Azure, Free offer	Schema `carteira` (3 tabelas)	1 DTU; 32 GB; pausa automática
MongoDB Atlas	M0 Free	Base `investlab_ativos`, coleção `ativos`	512 MB; conexões limitadas
Docker Hub / GitHub	Público	Imagens versionadas; código, Actions e Dependabot	Nenhum segredo nos artefatos

7.2 Mapeamento de blocos e configuração

Bloco	Infraestrutura	Variáveis de ambiente (valores nunca versionados)
mfe-*	Static Web Apps	`VITE_BFF_URL` (URL do gateway), `VITE_MFE_ATIVOS_URL`, `VITE_MFE_CARTEIRA_URL`
bff-web	Container Apps	`MS_ATIVOS_URL`, `MS_CARTEIRA_URL`, `FN_SIMULACAO_URL`, `FN_SIMULACAO_KEY`, `CORS_ORIGINS`
ms-ativos	Container Apps + Atlas + Service Bus	`MONGODB_URI`, `SERVICEBUS_CONNECTION`, `SERVICEBUS_TOPICO`
ms-carteira	Container Apps + Azure SQL	`AZURE_SQL_CONNECTION`
fn-simulacao	Function App + Service Bus	`SERVICEBUS_CONNECTION`, `SERVICEBUS_TOPICO`, `SERVICEBUS_ASSINATURA`, `MS_CARTEIRA_URL`

Ambiente local: `docker-compose.yml` sobe MongoDB, Azure SQL Edge, os dois microserviços e o BFF; sem variáveis, os serviços usam repositórios em memória e publicação de eventos em log, o que permite demonstrar o fluxo completo sem custos.

8. Conceitos Transversais

8.1 Clean Architecture + Vertical Slice

Cada container Node segue a mesma organização. A **Clean Architecture** define as camadas e a regra das dependências (código aponta para dentro, em direção às regras de negócio); a **Vertical Slice Architecture** define que cada caso de uso é uma pasta autocontida em `src/features`, com endpoint (adaptador de entrada), handler (caso de uso) e teste. Frameworks (Express, zod), drivers (MongoDB, mssql, Service Bus) e o SDK do Azure Functions ficam nas bordas.

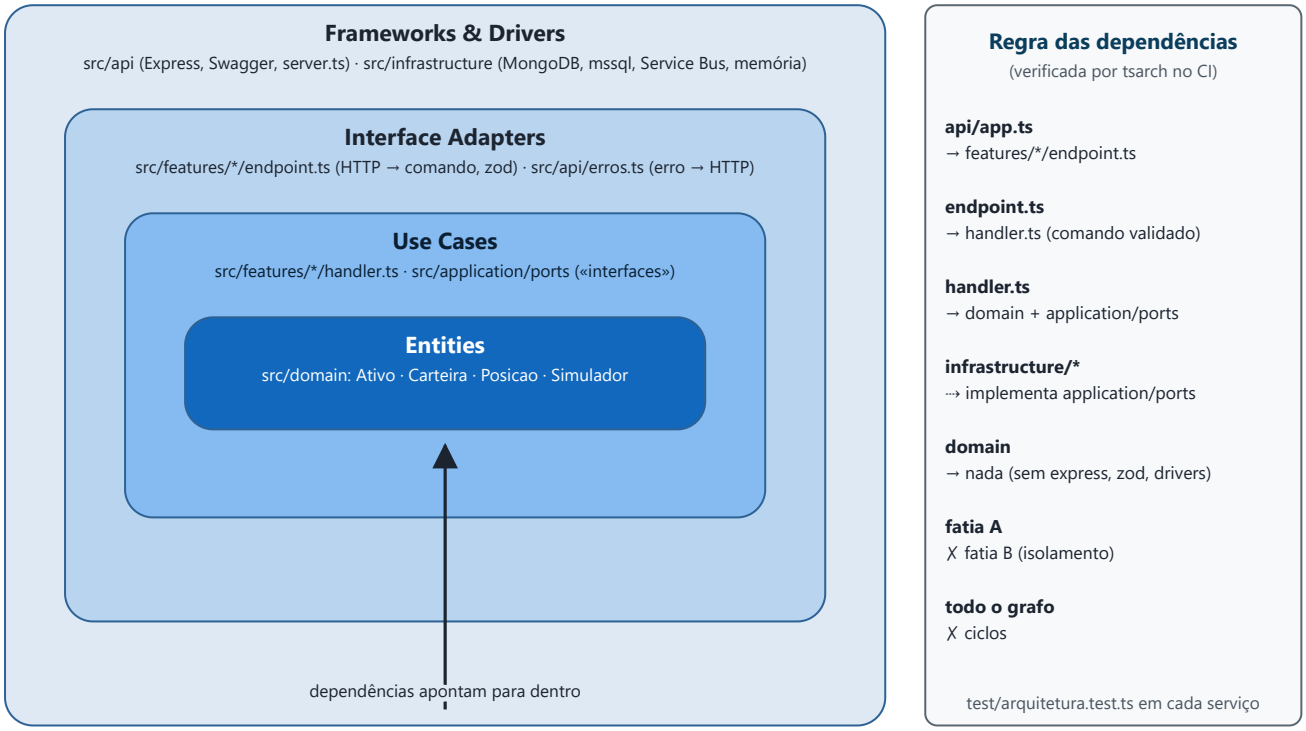


Figura 14 - Regra das dependências da Clean Architecture aplicada à estrutura de pastas de cada serviço (anéis concêntricos) e as regras que o teste de arquitetura impõe. Nenhuma seta sai de `domain`; `features` nunca aponta para `infrastructure`.

```
services/ms-ativos/
├── src/
│   ├── domain/              Ativo.ts · Dinheiro.ts · eventos.ts · erros.ts   (Entities, sem imports
externos)
│   ├── application/ports/   AtivoRepository.ts · EventPublisher.ts          (Output Ports)
│   ├── features/            uma fatia vertical por caso de uso
│   │   ├── cadastrar-ativo/   endpoint.ts · handler.ts · handler.test.ts
│   │   ├── listar-ativos/     endpoint.ts · handler.ts · handler.test.ts
│   │   ├── obter-ativo/       …
│   │   ├── atualizar-cotacao/ … (publica o evento)
│   │   └── remover-ativo/     …
│   ├── infrastructure/      mongo/ · mensageria/ · memoria/  (adaptadores que implementam as portas)
│   └── api/                 app.ts (composition root) · server.ts · openapi.ts · erros.ts
├── test/arquitetura.test.ts   regras tsarch (estilo ArchUnit)
└── Dockerfile · package.json · tsconfig.json · vitest.config.ts
```

Aplicação dos princípios SOLID vistos em aula: **SRP** (endpoint só traduz HTTP; handler só coordena; entidade só protege invariantes), **OCP** (nova funcionalidade = nova pasta em `features`, registrada no composition root), **LSP** (`InMemoryAtivoRepository` e `MongoAtivoRepository` são intercambiáveis nos testes e no runtime), **ISP** (portas pequenas por capacidade: `EventPublisher` separado de `AtivoRepository`), **DIP** (handlers recebem interfaces pelo construtor; adaptadores dependem das portas).

8.2 Testes unitários de arquitetura

Cada serviço possui `test/arquitetura.test.ts` executado junto com os testes unitários (`npm test`) e no CI. A biblioteca `tsarch` (equivalente ao ArchUnit/ArchUnitNET apresentados em aula, para TypeScript) analisa o grafo de imports e falha o build se uma regra for violada. A verificação negativa foi feita: um import de `infrastructure` dentro de `domain` faz a suíte falhar.

Regra	O que garante	Serviços
`domain` não depende de `application|features|infrastructure|api`	Entidades isoladas (anel interno)	ms-ativos, ms-carteira, fn-simulacao
`domain` não depende de `express|mongodb|mssql|@azure|zod`	Regras de negócio sem frameworks/drivers	ms-ativos, ms-carteira, fn-simulacao
`application` depende apenas de `domain`	Portas estáveis	ms-ativos, ms-carteira, bff-web
`features` não dependem de `infrastructure|api`	Casos de uso dependem de abstrações (DIP)	ms-ativos, ms-carteira, bff-web
Fatia não depende de outra fatia	Isolamento das vertical slices	ms-ativos (5 fatias), ms-carteira (6 fatias)
`infrastructure` não depende de `features|api`	Adaptadores só implementam portas	ms-ativos, ms-carteira
BFF não importa drivers de banco	BFF sem banco (padrão BFF + Database per Service)	bff-web
Grafo livre de ciclos	Modularidade	todos

```
// services/ms-ativos/test/arquitetura.test.ts (trecho)
it("features (casos de uso e endpoints) não dependem de infrastructure nem de api", async () => {
  const regra = filesOfProject()
    .inFolder("features")
    .matchingPattern("^(?!.*\\.test\\.ts$).*$")
    .shouldNot().dependOnFiles().matchingPattern("(infrastructure|api)/");
  expect(await regra.check()).toEqual([]);
});
it("não há ciclos de dependência no serviço", async () => {
  expect(await filesOfProject().inFolder("src").should().beFreeOfCycles().check()).toEqual([]);
});
```

Resultado atual da suíte: **45 testes** (24 unitários de domínio/casos de uso + 21 de arquitetura) passando em ms-ativos (17), ms-carteira (15), bff-web (6) e fn-simulacao (7).

8.3 Segurança e privacidade

- Única entrada pública é o API Gateway (CORS restrito ao domínio do shell; throttling). Microserviços e Functions recebem chamadas apenas do BFF/Function (rede interna e chave de função `x-functions-key`).
- Autenticação e autorização por identidade (RF01, Q06) implementadas no gateway (JWT authorizer) na Entrega 3; o `usuarioId` passa a vir do token, nunca do cliente.
- HTTPS em todas as interfaces publicadas; TLS nas conexões com Atlas e Azure SQL.
- Segredos somente em variáveis de ambiente/App Settings (`.env` e `local.settings.json` no `.gitignore`); repositórios públicos e imagens Docker não contêm credenciais; Key Vault previsto.
- Validação de entrada com zod em todos os endpoints e Functions antes de aplicar regras ou persistir.

8.4 Eventos e tratamento de dados externos

- **Contrato:** `investlab.ativos.cotacao-atualizada` v1 (JSON Schema em `contracts/events`); mudanças incompatíveis geram v2 e nova assinatura.
- **Semântica at-least-once:** o produtor persiste antes de publicar; o consumidor (`atualizar-precos`) é idempotente, então reentregas são seguras.
- **Falhas:** exceção no trigger → *abandon* → reentrega (até 10×) → dead-letter; alarmes sobre a DLQ.
- **Fontes externas** continuam isoladas no ms-ativos (futuro Timer trigger de coleta); falha não substitui a última base válida.

8.5 Regras de cálculo financeiro

Cálculos ficam concentrados em código puro: `Posicao` (preço médio ponderado, arredondado em centavos), `Carteira` (patrimônio e investido) e `Simulador` (juros compostos mensais com aporte, três cenários ±2 p.p.). Toda execução carrega parâmetros e `dataReferencia`, permitindo reprodução e auditoria (Q01, Q07).

8.6 Erros e resiliência

Situação	Comportamento
Entrada inválida	400 `ENTRADA_INVALIDA` com detalhes do zod.
Regra de domínio violada	422 `REGRA_DE_DOMINIO` (ex.: data de referência anterior).
Recurso inexistente / conflito	404 `NAO_ENCONTRADO` / 409 `CONFLITO`.
Upstream indisponível no BFF	Fonte vira `null` e `fontes.x = "rejected"` no agregado; proxies retornam 502 `UPSTREAM_INDISPONIVEL`; timeout 5 s.
Remote do microfrontend fora do ar	*Error boundary* no shell exibe aviso; demais módulos seguem.
Erro interno	500 `ERRO_INTERNO` sem vazar detalhes; log estruturado.

8.7 Observabilidade

- Logs estruturados em JSON (`nivel`, `operacao`, identificadores) em todos os serviços; Application Insights nas Functions e Container Apps.
- Métricas de duração de simulação (`duracaoMs`) e de agregação; contagem de mensagens na assinatura e na DLQ.
- `GET /health` em cada serviço para liveness; `dataReferencia` exposta como indicador de frescor dos dados.

8.8 API first e documentação viva

Todo serviço expõe `/openapi.json` e Swagger UI em `/docs`; o gateway é descrito em OpenAPI (`gateway/openapi-gateway.yaml`); eventos em JSON Schema. Este documento é gerado de `docs/arc42-investlab.html` (Mermaid), versionado junto ao código.

9. Decisões Arquiteturais

As decisões seguem o formato ADR apresentado em aula: título, contexto, decisão, **estado** e consequências (positivas e negativas). As decisões da Entrega 1 são mantidas, substituídas ou reafirmadas explicitamente.

ID	Decisão	Contexto	Consequências	Estado
ADR-01	Adotar monólito modular	Entrega 1: complexidade operacional limitada.	Menor custo inicial; não atende aos estilos exigidos (microserviços, BFF, EDA).	Substituída por ADR-08
ADR-02	React + Next.js no frontend	Aplicação web única.	Boa DX; não permite deploy independente por módulo.	Substituída por ADR-13
ADR-03	Python + FastAPI no backend	Domínio com estatística e cálculo.	Facilita motores analíticos; obriga duas linguagens, pois o BFF deve ser Node.	Substituída por ADR-09
ADR-04	PostgreSQL como persistência	Dados relacionais.	Modelo consistente; incompatível com Database per Service e com os bancos exigidos.	Substituída por ADR-10
ADR-05	Isolar fontes externas	B3, CVM, BCB, Tesouro e FGC variam em formato e disponibilidade.	Mudanças concentradas no ms-ativos (ingestão).	Aceita (mantida)
ADR-06	Registrar premissas e data de referência	Simulações e indicadores precisam ser reproduzíveis.	Rastreabilidade; `dataReferencia` em todo contrato e no evento.	Aceita (mantida)
ADR-07	Não executar ordens	Produto delimitado como plataforma analítica.	Reduz escopo e riscos regulatórios.	Aceita (mantida)
ADR-08	Microserviços por capacidade (ativos, carteira) + BFF + API Gateway	A disciplina exige os estilos; ativos e carteira têm ciclos de mudança e modelos de dados distintos; o frontend precisa de um único ponto de entrada.	+ Deploy e escala independentes; + entrada única com segurança centralizada; + payload otimizado pela web. − Mais partes para operar e monitorar; − latência extra (gateway + BFF); − testes de integração mais complexos.	Aceita
ADR-09	TypeScript/Node.js 22 em todos os containers	O BFF deve ser Node (enunciado); Functions em Node já foram praticadas; equipe de 4 pessoas.	+ Uma linguagem, um toolchain (Vitest, tsarch), tipos compartilháveis por contrato; + menor carga cognitiva. − Perde Pandas/SciPy para estatística avançada (RF05/RF14 exigirão bibliotecas JS ou uma Function Python futura).	Aceita
ADR-10	Database per Service: MongoDB Atlas para ativos, Azure SQL para carteira	Ativos são documentos consultados por tipo; carteira exige integridade referencial e cálculo transacional; enunciado da Entrega 3 fixa os dois bancos.	+ Modelo adequado a cada domínio; + isolamento de falhas e esquemas. − Sem join entre bases (referência lógica por ticker); − consistência eventual; − dois drivers e dois backups.	Aceita

ADR-11	EDA com Azure Service Bus (tópico + assinatura) para propagar cotações	Mudança de cotação precisa refletir nas carteiras sem acoplar ms-ativos a ms-carteira; grupo já operou Service Bus + trigger.	+ Desacoplamento, reentrega e DLQ; + novos consumidores sem alterar o produtor. − Consistência eventual visível ao usuário; − depuração distribuída; − custo fixo do plano Standard.	**Aceita**
ADR-12	Serverless (Azure Functions) para simulação e consumo de eventos	Simulação é CPU sob demanda e picos; consumidor de eventos não justifica container dedicado.	+ Pagamento por execução, escala automática; + trigger nativo de Service Bus. − Cold start; − limite de tempo de execução; − dependência de provedor.	**Aceita**
ADR-13	Microfrontend com React + Vite + Module Federation (shell + 2 remotes) no Static Web Apps	Módulos "Ativos" e "Carteira" evoluem separadamente; atividade da Semana 4 já publicou SWA.	+ Deploy independente, isolamento de falhas por módulo. − Governança de versões de React compartilhado; − configuração inicial maior; − consistência de UI exige design system (CSS no shell).	**Aceita**
ADR-14	Clean Architecture + Vertical Slice + testes de arquitetura (tsarch) em cada serviço	Regras financeiras devem ficar independentes de framework/banco; mudanças chegam por funcionalidade; a estrutura precisa resistir à erosão.	+ Testabilidade (fakes em memória), mudanças localizadas, regra verificada no CI. − Mais arquivos por funcionalidade (endpoint/handler/teste); − risco de over-engineering em CRUDs simples (mitigado: sem camadas extras de DTO/mapper quando não há variação).	**Aceita**
ADR-15	Azure como nuvem principal; AWS apenas para o API Gateway; Docker Hub para imagens	Azure for Students cobre Functions, SWA, Service Bus e SQL; o enunciado da Entrega 3 pede AWS Gateway e Docker Hub.	+ Reuso do que a equipe já praticou; + atende ao enunciado. − Multi-cloud introduz latência entre provedores e duas consoles/IAMs; alternativa documentada: Azure API Management (Consumption).	**Aceita**

10. Requisitos de Qualidade

A árvore de qualidade (estilo ATAM, como apresentado em aula) organiza as metas da seção 1.2 em categorias e cenários mensuráveis. Os valores são critérios de avaliação arquitetural; medições reais serão feitas na Entrega 3 e na avaliação ATAM da Semana 13.

10.1 Árvore de qualidade

Categoria	Qualidade (prioridade)	Descrição	Cenários
Funcionalidade	Correção (1), Precisão (1)	Resultados determinísticos; centavos inteiros; fórmulas verificadas por testes.	Q01, Q02
Usabilidade	Facilidade de uso (2)	Comparação simples sem treinamento.	Q03
Eficiência	Desempenho (3)	Agregação paralela; simulação serverless.	Q04, Q08
Confiabilidade	Confiabilidade dos dados (4), Disponibilidade (7)	Última base válida; degradação graciosa; reentrega de eventos.	Q05, Q09
Segurança	Confidencialidade (5)	Acesso só pelo gateway autenticado; segredos fora do código.	Q06
Manutenibilidade	Modificabilidade (6), Testabilidade (6)	Fatias independentes; regra das dependências verificada.	Q07, Q10

10.2 Cenários de qualidade

ID	Qualidade	Cenário (estímulo → resposta)	Critério
Q01	Correção	Mesmas entradas, mesma versão e mesma base de referência em duas execuções da simulação.	100% de igualdade (teste `Simulador.test.ts`).
Q02	Precisão	Aportes com preços diferentes atualizam o preço médio da posição.	Sem divergência de centavos (teste `Carteira.test.ts`).
Q03	Usabilidade	Usuário iniciante cadastra um ativo, cria carteira, registra aporte e simula.	Até 10 minutos, sem treinamento.
Q04	Desempenho	`GET /aggregated-data` com os três upstreams saudáveis.	95% em até 3 s (chamadas em paralelo, timeout 5 s).
Q05	Confiabilidade de dados	Chega cotação com data de referência anterior à vigente.	Rejeitada com 422; base válida intacta; nenhum evento publicado (teste `atualizar-cotacao`).
Q06	Segurança	Cliente tenta acessar ms-ativos/ms-carteira/Function diretamente pela internet.	Sem rota pública; apenas o gateway responde; chamadas sem JWT negadas (E3).
Q07	Rastreabilidade	Usuário revisita uma carteira após várias atualizações de preço.	Transações e `dataReferencia` armazenadas; histórico reproduz o preço médio.

Q08	Escalabilidade	Pico de 100 simulações simultâneas.	Functions escalam automaticamente; nenhum container precisa ser redimensionado.
Q09	Disponibilidade / resiliência	ms-carteira indisponível durante entrega de evento; ms-ativos indisponível durante o dashboard.	Evento reentregue até sucesso ou DLQ (sem perda); dashboard responde com `ativos: null` (teste `aggregated-data`).
Q10	Modificabilidade	Adicionar a funcionalidade "proventos" ao ms-ativos.	Nova pasta em `features` + registro no composition root; nenhuma fatia existente alterada; testes de arquitetura continuam verdes.

11. Riscos e Dívida Técnica

Os riscos da Entrega 1 são mantidos onde ainda se aplicam e complementados pelos riscos introduzidos pela arquitetura distribuída. Devem ser revisitados a cada entrega.

ID	Risco	Impacto	Prob.	Mitigação
R01	Dependência de fontes externas	Alta	Média	Integrações isoladas no ms-ativos; validação; última base válida preservada por regra de domínio.
R02	Erro em fórmula financeira	Alta	Alta	Código puro em `domain` com testes unitários de casos conhecidos; centavos inteiros.
R03	Dados desatualizados	Média	Alta	`dataReferencia` em todo contrato e aviso na interface.
R04	Crescimento de escopo	Alta	Alta	Fatias futuras listadas em 11.1; escopo da Entrega 3 fixado pelo enunciado.
R05	Exposição indevida de carteira	Alta	Média	Gateway como única entrada; JWT na Entrega 3; `usuarioId` do token.
R06	Consistência eventual confundir o usuário (novo)	Média	Alta	Exibir "atualizada em"; consumidor idempotente; monitorar atraso da assinatura.
R07	Cold start de Functions e Container Apps (novo)	Média	Alta	Simulação leve; aceitar latência inicial; opção de instância mínima = 1 se necessário.
R08	Limites e pausas dos planos gratuitos (novo)	Média	Alta	Carga acadêmica; health checks; documentação de limites na seção 7.1.
R09	Multi-cloud (AWS Gateway + Azure) e duplicidade de IAM (novo)	Média	Média	Gateway restrito a proxy; alternativa APIM documentada; credenciais mínimas.
R10	Depuração distribuída (BFF, eventos, Functions) (novo)	Média	Média	Logs JSON com identificadores; Application Insights; DLQ monitorada; teste ponta a ponta local sem nuvem.
R11	Versionamento de React compartilhado entre shell e remotes (novo)	Baixa	Média	`shared: ["react","react-dom"]` com mesma versão fixada no monorepo.

11.1 Dívida técnica prevista

- Autenticação (RF01) e autorização por usuário ainda não implementadas: o `usuarioId` é informado pelo cliente até o JWT authorizer da Entrega 3.
- Coleta automática de cotações (RF19) depende de um Timer trigger e dos adaptadores de B3/CVM/BCB/Tesouro/FGC, ainda não codificados.
- Fatias futuras já posicionadas: proventos e indicadores (ms-ativos), renda fixa/tributação/FGC e backtest (fn-simulacao), perfil e laboratório de estratégias (ms-carteira), Box Plot (mfe-ativos).
- Repositório SQL faz *delete* + *insert* das posições a cada salvamento; adequado ao volume acadêmico, mas deve evoluir para *upsert* por posição.
- Sem *outbox pattern*: entre persistir e publicar pode haver falha; aceitável agora, candidato a melhoria.
- Observabilidade centralizada (dashboards, alertas de DLQ) e pipeline de CI/CD completo ficam para a Entrega 3.

Referências

Materiais produzidos pelo grupo:

- InvestLab - Documentação de Arquitetura (Entrega 1, 10/09/2026) - Arc42 parcial, C4 níveis 1-4, DER e Canvas.
- InvestLab_C4.md - modelo C4 (Structurizr DSL) da Entrega 1; MeuSistema.md - Arc42 seções 1-3.
- Evidências das atividades formativas: Azure Function (GetStatus), Azure Service Bus + ServiceBusQueueTrigger, Static Web Apps + Azure Functions, Cloud Native Landscape.
- Repositório `investlab` (monorepo): serviços, microfrontends, contratos, gateway, `docs/`.

Material da disciplina (Prof. Manoel Valerio da Silveira Neto, PUCPR, 2026/2): slides de Documentação para Arquitetura (C4 Model e UML), Arc42, Software Architecture Canvas, Fundamentos de Computação em Nuvem, Serverless/Cloud Native, Microfrontend, Padrões e Estilos Arquiteturais (API Gateway, BFF, Microservices, Database per Service, CQRS), Event-Driven Architecture, Clean Architecture, Vertical Slice, SOLID e Clean Code, Testes de Arquitetura (ArchUnit).

Referências arquiteturais oficiais:

- arc42 Documentation - https://docs.arc42.org/ ; arc42 Download - https://arc42.org/download/ ; Software Architecture Canvas - https://canvas.arc42.org/
- C4 Model - https://c4model.com/ ; Diagrams - https://c4model.com/diagrams ; Notation - https://c4model.com/diagrams/notation
- Newman, S. Backends For Frontends - https://samnewman.io/patterns/architectural/bff/
- Martin, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. 2017.
- Fowler, M. Micro Frontends; Event Sourcing; CQRS - https://martinfowler.com/
- Microsoft Learn - Azure Functions (Service Bus trigger), Azure Static Web Apps, Azure Service Bus, Azure SQL Database.
- AWS - API Gateway HTTP APIs (OpenAPI import); MongoDB Atlas; tsarch - https://github.com/ts-arch/ts-arch ; ArchUnitNET - https://github.com/TNG/ArchUnitNET

Declaração de uso de Inteligência Artificial Generativa

Durante a preparação desta documentação de arquitetura e do código-base que ela descreve, os autores usaram Claude (Anthropic, via Claude Code, outubro de 2026) para estruturar o documento a partir da Entrega 1 e do material da disciplina, gerar o esqueleto do código, os testes e os diagramas em Mermaid. Após usar essa ferramenta, os autores revisaram e editaram o conteúdo conforme necessário e assumem total responsabilidade pelo conteúdo.

> **Observação sobre o recorte de seções:** o site oficial do arc42 apresenta 12 seções. O enunciado desta atividade solicita 11; por isso, esta entrega segue deliberadamente as seções 1 a 11, sem incluir uma seção 12 (Glossário) como parte da nota. Os termos técnicos estão definidos ao longo do texto e na seção 4.2.